

Sleep-Time Compute 쉬운 설명본

어려운 Study paper 를 그림과 예로 다시 읽는 안내서

읽는 법 이 문서는 원 Study 의 결론을 바꾸지 않고, 질문→세 문장→직관→예/반례→그림→대안→미증명 지점 순서로 풀어 쓴다. 파란색 training 용어는 별도 Background paper 의 해당 개념으로 연결된다. 주장 ID 와 figure ID 는 원 Study 와 동일하다.

Contents

E01 · Sleep-time compute 란 무엇이며 왜 training 인가	5
이 권이 답할질문	5
먼저 세 문장	5
직관	5
예와 반례	6
그림 읽기	7
대안과 비교	7
아직 모르는것	8
더 깊이 읽기	8
E02 · 어떤 문제를풀려는가	10
이 권이 답할질문	10
먼저 세 문장	10
직관	10
예와 반례	11
그림 읽기	11
대안과 비교	12
아직 모르는것	13
더 깊이 읽기	13
E03 · 뇌의 수면에서 agent memory 까지의계보	14
이 권이 답할질문	14
먼저 세 문장	14
직관	14
예와 반례	15
그림 읽기	15
대안과 비교	16
아직 모르는것	16
더 깊이 읽기	17
E04 · Sleep 말고 가능한 최신대안들	18
이 권이 답할질문	18

먼저 세 문장	18
직관	18
예와 반례	19
그림 읽기	20
대안과 비교	20
아직 모르는것	21
더 깊이 읽기	21
E05 · Sleep phase 안에서 실제로 무엇을 계산하는가	23
이 권이 답할질문	23
먼저 세 문장	23
직관	23
예와 반례	24
그림 읽기	25
대안과 비교	26
아직 모르는것	26
더 깊이 읽기	27
E06 · 강한 대안과 정면 비교하면 무엇이 남는가	28
이 권이 답할질문	28
먼저 세 문장	28
직관	28
예와 반례	29
그림 읽기	29
대안과 비교	31
아직 모르는것	31
더 깊이 읽기	32
E07 · 논문 · 제품 · negative evidence 는 어디까지 왔는가	33
이 권이 답할질문	33
먼저 세 문장	33
직관	33
예와 반례	34
그림 읽기	35
대안과 비교	35
아직 모르는것	35
더 깊이 읽기	36
E08 · 정말 promising 하며 주류가 될까	37
이 권이 답할질문	37
먼저 세 문장	37

직관	37
예와 반례	38
그림 읽기	38
대안과 비교	40
아직 모르는것	40
더 깊이 읽기	40
E09 · Sleep-time scaling law 는 어떻게 세워야하는가	41
이 권이 답할질문	41
먼저 세 문장	41
직관	41
예와 반례	42
그림 읽기	43
대안과 비교	43
아직 모르는것	44
더 깊이 읽기	44
E10 · Wake·sleep·long-term memory infra 는 어떻게나뉘는가	47
이 권이 답할질문	47
먼저 세 문장	47
직관	47
예와 반례	48
그림 읽기	49
대안과 비교	49
아직 모르는것	51
더 깊이 읽기	51
E11 · Memory device 회사에 생기는기회	52
이 권이 답할질문	52
먼저 세 문장	52
직관	52
예와 반례	53
그림 읽기	54
대안과 비교	55
아직 모르는것	56
더 깊이 읽기	56
E12 · 무엇을 측정하면 이 가설을 믿거나 버릴 수있는가	57
이 권이 답할질문	57
먼저 세 문장	57
직관	57

예와 반례	58
그림 읽기	59
대안과 비교	60
아직 모르는것	60
더 깊이 읽기	60
문서 경계와 사용법	61

E01 · Sleep-time compute 란 무엇이며 왜 training 인가

이 권이 답할질문

“모델이 잔다” 는 표현을 잠시 버리고도 sleep-time compute 를 정확히 정의할수 있을까? 답은 **사용자 응답이 끝난 뒤 계산한다**는 시간 조건과, 그 계산이 **다음 wake 에서 재사용될 상태를 바꾼다**는 상태 조건을함께 보는 것이다. 단순한 야간 batch job, 로그 압축, model checkpoint 복사는 첫 조건만 만족할 뿐이다. 반대로 응답 도중의 chain-of-thought 나 긴추론은 계산량이 커도 다음 episode 에 남지 않으면 두 번째 조건을 만족하지못한다.

이 권은 Study 의 operational definition 과 training map 을 쉬운 말로 연결한다. 핵심 근거는 **STC-C005 · synthesis · confidence high**, **STC-C006 · synthesis · confidence high**, **STC-C015 · synthesis · confidence high** 이다.

먼저 세 문장

1. Sleep-time compute 는 latency-critical response 뒤로 미룬계산이 미래의 reusable state 를 바꾸는 lifecycle 이다. “sleep” 이라는이름보다 이 두 조건이 중요하다.
2. 바뀌는 state 는 foundation weight 만이 아니다. adapter, recurrent state, text/vector memory, temporal graph, summary, index 도 가능하므로언제 갱신하는가와 어디에 기억하는가는 서로 독립된축이다.
3. state 를 경험에 맞춰 개선하려면데이터→순전파→손실→역전파→옵티마이저갱신 중 일부 또는 전부가 필요하다. 그래서 sleep 은 inference 의 부속기능이 아니라 작은 training system 에 가깝다.

한 줄 요약

Wake 가 “지금 답하기” 라면 sleep 은 “다음번에 더 잘 답하도록 재사용상태를 고치기” 다.

직관

두 개의 경계선을 그리면된다

첫 번째 경계는 **응답 SLA** 다. 사용자가 기다리는 동안 반드시 끝나야하는 계산이 wake path 이고, 사용자가 답을 받은 뒤 실행해도 되는 계산이 deferred path 다. 두 번째 경계는 **episode** 다. 계산 결과가 현재응답에서 사라지는지, 다음 대화 · 다음 task · 다음 날에도 재사용되는지를나눈다.

계산	응답 뒤 실행	다음 episode 에 재사용	STC 인가
응답 중 더 오래 생 각하기	아니오	대개 아니오	아니오
로그를 cold storage 로 복사	예	행동을 바꾸지 않음	보통 아니오
대화에서 stable preference 를 추 출해 memory store 에 기록	예	예	예, external-memory STC
replay 로 adapter 를 갱신 하고 새 version 을 배포	예	예	예, parametric STC

계산	응답 뒤 실행	다음 episode 에 재사용	STC 인가
search index 를 rebuild 해 이후 retrieval 을 개선	예	예	예, state/index STC

직관

“밤에 들었다”가 아니라 “다음번 행동을 바꾸는 상태가 남았다”가 판정기준이다. 이 정의를 쓰면 마케팅 용어가 달라도 dreaming, reflection, synthesis, consolidation 을 같은 좌표계에 놓을 수 있다.

Training 은 weight update 보다 넓다

Training 을 “거대한 pretraining cluster 에서 모든 weight 를 바꾸는 일”로만 보면 external-memory sleep 은 training 이 아닌 것처럼 보인다. 그러나 시스템 관점의 training 은 **경험으로부터 미래 행동을 개선할 state transformation 을 선택하는 일**이다. text summary 를 새로 만들 때도 어떤 episode 를 넣고 뺄지, 충돌을 어떻게 합칠지, 품질을 어떤 evaluator 로 판단할지에 objective 가 숨어 있다.

Parametric sleep 은 그 objective 를 gradient 로 직접 최적화한다. External sleep 은 LLM extraction, deduplication, clustering, graph update 처럼 더이산적인 procedure 를 쓴다. 둘 다 오류가 누적될 수 있고 validation·rollback·provenance 가 필요하다. 차이는 “학습인가 아닌가”보다 학습 결과가 어느 **medium** 에 저장되고 얼마나 되돌리기 쉬운가다.

예와 반례

예 1 — 외부 기억공고화

한 사용자가 여러 대화에서 “회의 요약은 결론부터, 표는 싫다”고 반복했다고 하자. sleep job 은 episode 들을 모아 중복을 제거하고, 일시적 요청과 stable preference 를 구분하고, “결론 우선 · 표 지양”이라는 durable memory 를 만든다. foundation weight 는 고정이지만 다음 대화에서 retrieval 되어 행동을 바꾸므로 STC 다. 장점은 원 episode 와 연결해 수정 · 삭제하기 쉽다는 점이다.

예 2 — adapter 로 선택적승격

한 공장 line 에서 반복되는 error code 와 복구 절차가 수천 번 재사용된다면, 매번 긴 문서를 retrieve 하는 비용보다 domain adapter 로 compile 하는 편이 유리할 수 있다. sleep job 은 검증된 record 만 골라 증류 또는 fine-tuning 으로 작은 delta 를 만들고, canary 평가를 통과한 뒤 활성화한다. 이것은 더 빠를 수 있지만 삭제 · 오염 · rollback 은 어려워진다.

반례 1 — 긴 test-time reasoning

한 질문에 100 배 많은 token 을 써서 답을 찾았지만 그 reasoning 이 버려졌다면 test-time compute 이지 STC 는 아니다. 같은 reasoning 에서 reusable strategy 를 추출해 다음 task 의 strategy memory 로 남겼을 때 비로소 lifecycle 의 sleep branch 가 된다.

Wake-test-sleep lifecycle

경계는 clock time이 아니라 response-critical dependency다

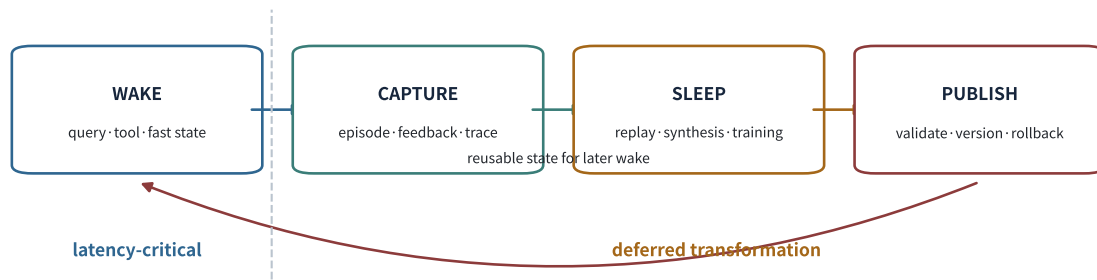


Figure 1: STC-F001. Wake-test-sleep lifecycle and reusable-state boundary. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

반례 2 — 무조건적인 nightly fine-tuning

매일 밤 모든 user log 로 LoRA 를 학습한다고 해서 좋은 STC 가 되는 것은 아니다. 잘못된 label, transient intent, privacy-sensitive record 까지 섞이면 다음날 성능이 악화된다. “뒤에서 학습했다” 는 사실은 mechanism 이고, 무엇을 admit 하고 무엇을 보존하며 어떻게 되돌릴지가 solution 이다.

주의

Sleep-time compute 는 자동으로 안전하거나 효율적이라는 뜻이 아니다. latency budget 을 sleep budget 으로 옮겼을 뿐, 잘못된 학습의 위험과 비용은 사라지지 않는다.

그림 읽기

이 그림은 pretraining/post-training 이후의 배포 lifecycle 을 wake 와 sleep 으로 다시 자른다. 왼쪽에서 wake 는 요청 처리, retrieval, fast-state update 를 담당한다. 오른쪽 sleep 은 replay·reflection·distillation·compaction 을 실행한다. 가운데 reusable-state boundary 가 가장 중요하다. 그 경계를 넘는 object 가 text 인지 graph 인지 adapter 인지 에 따라 storage 와 governance 가 달라진다.

두 번째 그림은 “시간” 과 “medium” 을 직교축으로 놓는다. wake-time weight update 도 가능하고 sleep-time external-memory update 도 가능하다. 따라서 “parametric vs external” 과 “wake vs sleep” 을 한 축으로 섞으면 안 된다. 실제 system 은 네 사분면을 동시에 쓸 수 있다: wake 중 recurrent state, 응답 후 summary, 주기적 graph compaction, 드물게 adapter promotion.

대안과 비교

Pretraining·post-training·test-time·sleep-time 의 역할

phase	대표 데이터	바뀌는 것	강점	약점
pretraining	대규모 일반 corpus	foundation weights	넓은 prior	매우 비싸고 최신성 낮음

phase	대표 데이터	바뀌는 것	강점	약점
post-training	instruction-preference-domain data	weights/adapter	행동 정렬	배포 전 분포에 묶임
wake/test-time learning	현재 con-text-feedback	activation/fast state/weight	즉시 적응	응답 latency와 interference
sleep-time learning	누적 episode-replay-derived data	external/parametric reusable state	batch·검증·공고화	staleness, pipeline 복잡도

Sleep의 고유 장점은 **현재 응답의 latency와 학습 계산을 분리**하고, 여러 episode를 한꺼번에 비교해 dedup·conflict resolution·evaluation할 수 있다는 점이다. 반면 즉시 반영이 필요한 정보에는 늦다. 그래서 최선의 framing은 네 phase 중 하나를 승자로 고르는 것이 아니라, 각 state를 어느 phase에서 갱신할지 결정하는 것이다.

Fixed와 session-specific weights

Model artifact를 foundation weights, shared domain adapter, user adapter, external memory, ephemeral state로 분리하면 위험이 보인다. foundation은 느리고 검증이 엄격해야 한다. user adapter는 빠르게 personalized될 수 있지만 tenant isolation과 version explosion이 생긴다. external memory는 설명 가능하지만 retrieval latency와 context budget을 쓴다. “모든 기억을 weight로” 또는 “모든 기억을 vector DB로”는 둘 다 극단이다.

아직 모르는것

- 어떤 deferred job이 실제 미래 utility를 만들었는지 측정하는 공통 benchmark가 없다.
- Google의 Language Models Need Sleep은 parametric consolidation의 직접 증거지만, 공개 실험이 곧 장기 fleet 운영·무제한 user별 weight 성장의 증거는 아니다.
- External-memory sleep은 이미 product 형태가 보이지만, derived summary가 원 evidence를 왜곡하지 않는다는 보장은 없다.
- “얼마나 자주 자야 하나”는 traffic, staleness, batching, interference의 함수이며 고정 주기로 해결되지 않는다.
- 같은 sleep budget을 retrieval 개선, synthetic data, adapter training 중 어디에 배분할지에 대한 universal policy가 없다.

핵심

이 분야의 첫 연구 질문은 “sleep이 가능한가?”가 아니라 “어떤 reusable state를 어떤 evidence와 objective로, 어느 cadence에, 얼마나 되돌릴 수 있게 바꿀 것인가?”다.

더 깊이 읽기

- 원 분석: 운영적정의와배포전·wake·sleep training map
- Training Background: Wake 상태, Sleep 상태, 손실, 역전파, 옵티마이저
- Claim route: **STC-C005**·synthesis·confidence high·**STC-C006**·synthesis·confidence high·**STC-C015**·synthesis·confidence high

두 개의 독립 축: update timing × memory medium

	Within-query	Post-response	Periodic sleep
Ephemeral / state	KV / recurrent state	session fast state	state compaction
External text · vector · graph	retrieval write	Letta / Mem0	Dreaming / graph rebuild
Parametric weights · adapter	TTT / neural memory	async adapter	replay / distill / refresh

같은 medium도 timing이 다르면 SLA · data horizon · validation이 달라진다

Figure 2: STC-F002. Update timing by memory-medium taxonomy. 원 Study 의저자 작성 합성 figure이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

E02 · 어떤 문제를 풀려는가

이 권이 답할 질문

Sleep-time compute 가 등장한 이유를 “AI 도 사람처럼 자야 해서” 라고 설명하면 문제를 놓친다. 실제 출발점은 배포된 agent 가 장기간 경험을 쌓을 때 생기는 다섯 병목이다: **context overflow, state staleness, catastrophic forgetting, finite capacity, governance debt**. 이 중 일부는 retrieval 이나 cache 로 해결되고, 일부는 background consolidation 이유이며, 일부는 아직 어떤 방식으로든 해결되지 않았다.

이 권은 문제분해를 바탕으로 **STC-C032** · author_claim · confidence high, **STC-C033** · synthesis · confidence high, **STC-C035** · author_claim · confidence high, **STC-C037** · synthesis · confidence high 을 설명한다.

먼저 세 문장

1. Long context 는 “많이 넣는 능력” 이지 “무엇을 오래 보존하고 갱신할지 결정하는 능력” 이 아니다. context window 가 커져도 오래된 episode 의 충돌 · 삭제 · 중요도 문제는 남는다.
2. 지속적으로 weight 나 memory 를 갱신하면 유한한 용량 예산 안에서 새 지식이 옛 지식의 접근 가능성을 손상시킬 수 있다. 정보가 parameter 안에 남아 있어도 행동으로 꺼내지 못하면 실용적으로는 잊은 것이다.
3. Sleep 은 여러 episode 를 batch 로 비교하고 정리할 기회를 주지만, synthetic replay 의 오류 · memory poisoning · 삭제 전파라는 새 문제도 만든다.

직관

문제 1 — Context 는 작업대이지 창고가 아니다

Context window 는 현재 reasoning 에 필요한 자료를 펼쳐 놓는 작업대다. 작업대가 커지면 더 많은 문서를 동시에 볼 수 있지만, 매번 모든 과거를 다시 펼치는 비용은 sequence 길이에 따라 증가한다. 또한 중요하지 않은 기록이 signal 을 가리고, 서로 충돌하는 사실의 시간 순서를 모델이 잘못 읽을 수 있다. 그래서 long context 와 long-term memory 는 보완 관계이지 대체 관계가 아니다.

문제 2 — 기억은 저장보다 선택이 어렵다

새 episode 를 모두 저장하면 memory 는 단조 증가한다. 검색 후보가 늘면서 index 비용, false retrieval, compaction 비용, privacy exposure 가 함께 증가한다. 반대로 aggressive summary 는 중요한 tail detail 을 잃는다. 어떤 record 를 admit · merge · supersede · evict 할지 결정하는 정책이 필요하다.

문제 3 — Weight 에는 주소가 없다

외부 DB 에서는 “이 문장은 어느 대화에서 왔는가” 를 pointer 로 남길 수 있다. Weight update 는 많은 example 을 gradient 로 섞기 때문에 특정 사실이 어느 parameter 에 얼마나 들어갔는지 역추적하기 어렵다. 삭제 요청이 오면 adapter 전체를 버리거나 원 dataset 에서 재학습해야 할 수도 있다. 따라서 parametric memory 는 빠른 access 를 얻는 대신 provenance 와 surgical delete 를 잃기 쉽다.

문제 4 — Continual learning 은 retention 과 plasticity 의 줄다리기다

지속학습은 새 경험에 적응해야 하지만 옛 능력을 지켜야 한다. update 를 크게 하면 plastic 하지만 망각이 커지고, 작게 하면 안정적이지만 새 정보가 안들어간다. 이를 파국적 망각과 stability-plasticity dilemma 라고 부른다. 단순히 parameter 를 늘리면 시간을 벌 수 있지만 capacity 와 serving cost 가 계속 자란다.

직관

도서관의 진짜 병목은 책장을 더 사는 일이 아니라, 중복판을 합치고 낡은 정보를 표시하고 출처를 남기며 필요한 책을 제때 찾는 일이다.

예와 반례

예 — “내 주소” 가 세 번 바뀐 사용자

2025 년, 2026 년 봄, 2026 년 여름 주소가 서로 다른 세 episode 에 있다. Naive vector search 는 query 와 가장 비슷한 과거 주소를 가져올 수 있다. Temporal graph 는 valid-time 과 transaction-time 을 붙여 최신 주소를 고를 수 있다. Summary 는 “현재 주소” 만 남겨 간단하지만, 왜 바뀌었는지와 과거 배송분쟁을 읽을 수 있다. Weight update 는 답을 빠르게 만들 수 있지만 어떤 주소가 학습되었는지 검증·삭제가 가장 어렵다.

예 — 새로운 error code 의 반복학습

공장 장비의 새 error code 가 매일 수백 번 등장한다. 단기에는 external memory 에 원 log 와 복구 절차를 저장하는 편이 안전하다. 사용 빈도와 안정성이 확인되면 domain adapter 로 승격할 수 있다. 이후 제조사 문서가 수정되면 adapter 를 rollback 하고 다시 build 해야 한다. 이 흐름은 sleep 을 “기억 저장” 이 아니라 **promotion pipeline** 으로 본다.

반례 — Long context 만능리기

과거 1 년의 모든 log 를 million-token context 에 넣을 수 있다고 해도 매 요청마다 읽을 이유는 없다. 비용·latency·irrelevant evidence 가 늘고, 삭제요청이 반영됐는지 확인하기도 어렵다. Long context 가 유용한 것은 선택된 evidence 를 깊게 읽을 때다. 선택·정리는 여전히 memory lifecycle 의 일이다.

반례 — 모든 것을 nightly replay

Generated replay 가 과거 distribution 을 완벽히 대표한다고 가정하면 현실을 과도하게 단순화한다. 생성 모델은 rare tail 을 빼먹거나 자기 오류를 반복할 수 있다. **STC-C035** · author_claim · confidence high 는 real-data support 없이 synthetic data 를 재귀적으로 학습할 때 distribution tail 이 붕괴할 수 있음을 경고한다.

그림 읽기

행은 해결해야 할 문제, 열은 대안이다. Long context 는 immediate access 에는 강하지만 durable update 와 delete 에는 약하다. External memory 는 provenance-rollback 이 강하지만 retrieval 과 context cost 가 있다. Adapter/weight 는 access latency 를 줄일 수 있지만 interference 와

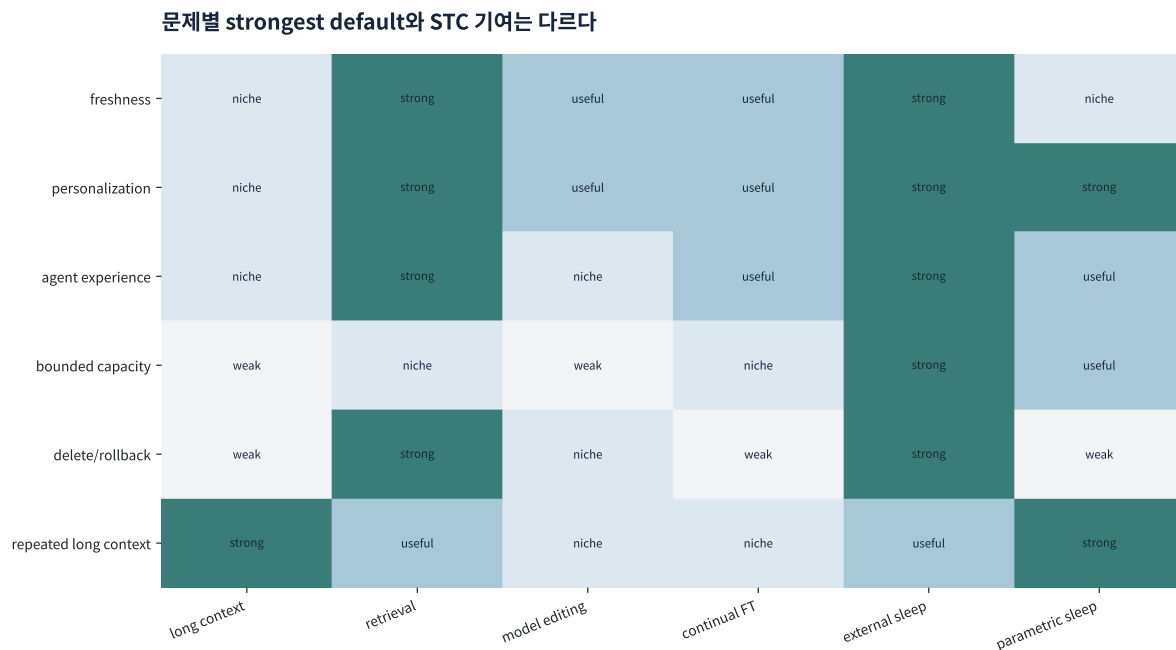


Figure 3: STC-F004. Problem-to-solution comparison matrix. 원 Study 의저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

governance가 어렵다. Sleep 은 독립 열이 아니라 이 방법들을 응답후 batch 로 조합하는 lifecycle 이다.

가로축은 memory load, 세로축은 retained utility 로 읽는다. 초반에는 memory 를 늘릴수록 utility 가 빠르게 증가하지만 어느 지점부터중복·간섭·검색 noise 가 커져 marginal utility 가 꺾인다. 이 “capacity knee” 는 이미 보편 law 로 입증된 값이 아니라 연구 가설이다. 중요한 것은 device capacity 만 늘리는 것이 아니라 knee 의 위치를 admission·compaction·tiering 으로 옮기는 것이다.

대안과 비교

근본 문제	sleep 이 제공하는 것	sleep 없이 가능한 강한 대안	판단 기준
context overflow	응답 후 요약·index rebuild	retrieval, recurrent state, compression	answer quality / token·latency
stale/conflicting facts	batch conflict resolution	temporal KG, system-of-record	temporal accuracy / delete
forgetting	replay·distillation·isolation	EWC, GEM, adapters, expansion	retained utility / interference
finite memory	compaction·promotion·eviction	rate-distortion policy	utility per byte / tail loss
governance	versioned sleep output	immutable event log	provenance / rollback time

Sleep 이 특히 유리한 곳은 한 episode 만 봐서는 판단할 수 없는일이다. 여러 기록에서 stable preference 를 찾아내거나, conflict 를 시간순으로 해결하거나, reuse 가 충분한 지식을 adapter 로 승격하는 일이다. 즉시반응이 중요하거나 source-of-truth 가 명확한 factual lookup 에는 external retrieval 이 더 단순하고 안전하다.

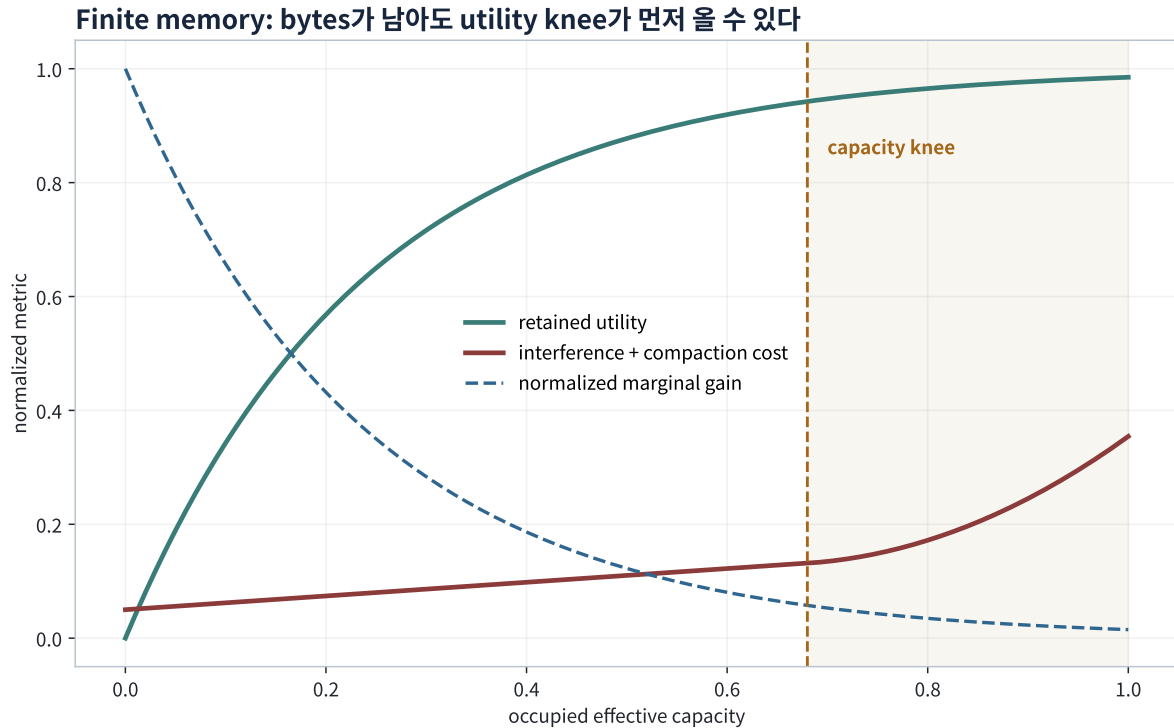


Figure 4: STC-F009. Finite-memory capacity knee. 원 Study 의 저자 작성합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

아직 모르는것

- Static memorization capacity 는 lifelong safe capacity 와 다르다. sequential interference·retrieval failure·delete overhead 를 함께 재는방법이 필요하다.
- Weight 안에 정보가 “encoded” 되었다는 probe 결과와 실제 answer 에서 “accessible” 하다는 결과 사이의 gap 을 어떻게 측정할지 합의가 없다.
- Synthetic replay 에서 real-data anchor 를 얼마나 유지해야 tail collapse 를 막는지 task 별 scaling law 가 없다.
- External memory poisoning 과 parametric poisoning 을 같은 benchmark 에서비교한 공개 증거가 부족하다.
- Memory capacity knee 가 workload, user 수, record entropy, device tier 에 따라 어떻게 이동하는지 측정되지 않았다.

주의

문제의 원인은 memory byte 부족 하나가 아니다. 잘못된 기억을 오래 · 빠르게제공하는 시스템은 큰 memory 를 가진 실패한 시스템이다.

더 깊이 읽기

- 원 분석: 다섯문제의 분해와 solution boundary
- Training Background: 지속학습, 파국적망각, 용량예산, 출처추적성, 일반화
- Claim route: **STC-C032** · author_claim · confidence high · **STC-C033** · synthesis · confidence high · **STC-C035** · author_claim · confidence high · **STC-C037** · synthesis · confidence high

E03 · 뇌의 수면에서 agent memory까지의 계보

이 권이 답할질문

Sleep-time compute 는 2026 년에 갑자기 생긴 이름인가, 아니면 오래된 연구흐름들이 한 lifecycle 에서 만난 것인가? 계보를 보면 적어도 네 갈래가 합쳐진다. 생물학적 memory consolidation, continual-learning replay, periodic knowledge compression, agent external memory 다. 이 계보를 알아야 “뇌를 모방했다” 는 약한 비유와 실제 algorithmic contribution 을 구분할 수 있다.

이 권은 1995 – 2026 계보를 풀고 **STC-C016** · author_claim · confidence high, **STC-C017** · author_claim · confidence high, **STC-C018** · author_claim · confidence high, **STC-C019** · author_claim · confidence high, **STC-C022** · author_claim · confidence high 을 연결한다.

먼저 세 문장

1. 생물학 연구가 준 핵심은 “수면” 이라는 단어가 아니라 **online acquisition** 과 **offline reorganization** 을 분리하고 서로 다른 memory system 이 상호작용한다는 가설이다.
2. Machine continual learning 은 sleep 이라는 이름 없이도 replay, regularization, parameter isolation, periodic compression 으로 망각을 줄여 왔다.
3. 최근 agent system 은 background synthesis 와 temporal memory maintenance 를 실제 product lifecycle 에 붙였고, neural-memory 연구는 weight/state 자체를 multi-timescale memory 로 해석하기 시작했다.

직관

계보 A — Complementary learning systems

빠르게 배우는 hippocampus 와 천천히 일반화하는 neocortex 를 분리한다는 관점은 한 저장소가 동시에 “즉시 기록” 과 “안정된 일반화” 를 잘하기 어렵다는 문제에서 출발한다. 낮에는 episode 를 빠르게 포착하고, offline 에는 재활성화해 느린 system 으로 통합한다. AI 로 옮기면 fast episodic store 와 slow parametric/generalized store 의 역할 분리가 된다.

계보 B — Replay

과거 example 을 그대로 저장하는 리플레이버퍼와, generator 가 pseudo-example 을 만드는 생성리 플레이가 있다. 둘 다 새 data 만 학습할 때 old task 가 사라지는 문제를 완화한다. 원 data replay 는 fidelity 가 좋지만 privacy-storage cost 가 있고, generated replay 는 저장을 줄이지만 generator bias 와 model collapse risk 가 있다.

계보 C — 보호와 격리

EWC 같은 정규화는 옛 task 에 중요한 parameter 를 덜 움직이게 한다. GEM 은 old example loss 가 악화되지 않도록 gradient 방향을 제약한다. Adapter 나 dynamic expansion 은 새 parameter 공간을 열어 옛 parameter 와 충돌을 줄인다. 이들은 모두 stability – plasticity 의 다른 해법이다.

계보 D — Periodic consolidation

Progress & Compress 는 active column 에서 새 task 를 배우고, 별도 phase 에서 knowledge base 로 distill 한다. 이는 명시적 “sleep” 이룬 전부터존재한 wake/periodic-consolidation 구조다. 오늘날의 STC 는 이를 long-lived agent, external memory, deployment scheduler, rollback 까지 확장한다.

직관

새 아이디어는 보통 재료가 아니라 조립법에 있다. Replay·distillation·adapter·graph 는 오래된 부품이고, STC 의 새로움은이들을 response-critical path 밖의 반복 lifecycle 로 묶는데 있다.

예와 반례

2022 PLOS sleep-like replay

두 task 를 순서대로 배운 spiking network 가 sleep-like replay 동안 joint synaptic representation 을 만들고 forgetting 을 줄였다는 결과는 biological analogy 와 computational mechanism 사이의 중요한 다리다. 다만 작은 two-task setting 의 결과를 그대로 billion-parameter LLM fleet 의 경제성으로 확대할 수는 없다. **STC-C016** · author_claim · confidence high 은 “망각 완화의 직접 증거” 이지 “LLM sleep scaling law” 의 증거가 아니다.

NREM/REM alternation 과 dreaming

Hippocampus – neocortex model 은 autonomous offline dynamics 와 NREM/REM-like regime 을 번갈아 새 정보 통합과 old knowledge 보호를연구했다. Perturbed and Adversarial Dreaming 은 wake, perturbed replay, adversarial dreaming 을 분리했다. 여기서 얻을 수 있는 design hint 는 phase 별 objective 가 다를 수 있다는 점이다. 하지만 인간 수면 stage 의 이름을 붙였다고 algorithm 이 생물학적으로 동일해지는 것은 아니다.

Deep Generative Replay

Generator 가 old data 와 비슷한 sample 을 만들고 solver 가 new+generated data 를 함께 학습한다. 이는 명시적 sleep 없이도 offline replay 의 핵심기능을 수행한다. 현대 sleep system 이 이 baseline 보다 우월하려면 단순 retention 외에 cadence, provenance, rollback, utility per cost 에서이겨야 한다.

반례 — “NREM 과 REM 을 구현했으니 더인간답다”

Stage label 은 evidence 가 아니다. NREM-like perturbation 과 REM-like adversarial objective 가 실제 어떤 ablation 에서 도움이 되었는지, 동일 compute 의 replay·regularization baseline 보다 나은지 확인해야 한다. 생물학적 plausibility 와 engineering optimality 는 독립된 질문이다.

그림 읽기

Timeline 은 1995 년 complementary learning systems 에서 2026 년 explicit sleep/product synthesis 까지 이어진다. 한 줄로 읽지 말고 네 층으로 본다. 아래에는 biological theory, 그 위에는

1995-2026: 이름보다 먼저 존재한 계보



Figure 5: STC-F003. 1995-2026 evidence chronology. 원 Study 의 저자 작성합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

continual-learning algorithm, 그위에는 neural/test-time memory, 맨 위에는 agent product 가 있다. 최근으로올수록 “망각 accuracy” 뿐 아니라 serving, user state, governance 가 연구문제에 들어온다.

대안과 비교

계보	보존 대상	대표 mechanism	현대 STC 에 남긴 것	해결하지 못한 것
biological consolidation	episode·representation	offline replay, phase alternation	fast/slow memory separation	fleet cost, delete
continual learning	task performance	replay, EWC, GEM, expansion distillation	retention baselines	agent state semantics
periodic compression	active knowledge	synthesis, graph maintenance	wake/consolidate lifecycle	long-term governance
agent external memory	user/entity facts		production background job	parametric generalization
neural memory	learned state	test-time update, multi-rate weights	state as memory	prefix reuse, serving maturity

이 표가 보여 주는 것은 sleep 이 단일 method 가 아니라 **research program** 이라는 점이다. 같은 lifecycle 안에서 어떤 workload 에는 raw replay, 어떤 workload 에는 graph merge, 어떤 workload 에는 adapter distillation 이 맞다.

아직 모르는것

- Biological stage 를 어느 수준까지 모방해야 engineering gain 이 있는지모른다. phase separation 자체가 충분할 수도 있다.

- Replay 가 retention 을 높여도 privacy, energy, rare-tail fidelity 를포함하면 최적이 아닐 수 있다.
- Multi-timescale weight memory 와 external temporal graph 를 함께 쓸 때어느 layer 가 system-of-record 인지 정립되지 않았다.
- 과거 연구는 짧은 task sequence 가 많아 수백 · 수천 sleep cycle 의 cumulative error 를 보여 주지 못한다.
- Consolidation 이 old knowledge 를 “보존” 하는지, 단지 benchmark answer 를재현하는 지 분리할 evaluation 이 필요하다.

핵심

Sleep-time compute 의 정당성은 인간이 잔다는 사실에서 나오지 않는다. Online path 와 offline path 를 분리했을 때 retention·utility·cost·governance 가 더 좋아진다는 비교 실험에서 나와야한다.

더 깊이 읽기

- 원 분석: 수면 ·continual learning·agent memory 의 시간순 계보
- Training Background: 생성리플레이, 리플레이버퍼, 공고화, 정규화, 동적확장
- Claim route: **STC-C016** · author_claim · confidence high · **STC-C017** · author_claim · confidence high · **STC-C018** · author_claim · confidence high · **STC-C019** · author_claim · confidence high · **STC-C022** · author_claim · confidence high

E04 · Sleep 말고 가능한 최신대안들

이 권이 답할질문

장기 기억과 continual adaptation 을 해결하려면 정말 sleep-time compute 가 최선인가? 비교 대상은 “아무것도 안 함” 이 아니라 2026 년 현재 강한대안들이다: long context 와 retrieval, temporal graph, recurrent state, model editing, LoRA/adapter, latent compilation, self-adaptation. Sleep 은 이들을 대체하는 새 layer 라기보다, 여러 대안을 **언제실행하고 어떤 state 로 승격할지** 정하는 orchestration 축에 가깝다.

이 권은 alternative frontier 를 바탕으로 **STC-C026 · author_claim · confidence high, STC-C027 · synthesis · confidence high, STC-C028 · synthesis · confidence high, STC-C029 · synthesis · confidence high, STC-C030 · synthesis · confidence high, STC-C031 · synthesis · confidence high** 을 비교한다.

먼저 세 문장

1. 사실의 최신성 · 출처 · 삭제가 중요하면 외부 기억과 retrieval 이 기본값이다. 이 문제를 weight update 로 풀 이유가 약하다.
2. 반복 사용되는 stable skill 이나 style 처럼 retrieval overhead 가 계속 발생하면 LoRA · adapter · latent compilation 으로 선택적 parametric promotion 을 고려할 수 있다.
3. Sleep 의 경쟁력은 한 method 의 peak accuracy 보다, evidence 를 accumulate 해 **admission** → **compaction** → **evaluation** → **versioning** 을 응답 밖에서 운영할 수 있다는 데 있다.

직관

대안 1 — Long context 와 RAG

RAG 는 mutable knowledge 를 model 밖에 두고 필요한 passage 만 context 에 넣는다. 원문을 citation 할 수 있고 update · delete 가 쉽다. 약점은 retrieval miss, context token, index maintenance 다. 하지만 factual memory 에 대한 governance baseline 으로 매우 강하다. Sleep 이 RAG 보다 낫다고 주장하려면 future query 에서 반복 retrieval 하는 비용보다 background synthesis 비용이 작고, 정확도 · 삭제 가능성도 유지되어야 한다.

대안 2 — Temporal graph

Entity 와 relation 에 validity interval 을 붙이면 “A 가 B 였다가 C 가 됨” 을 표현할 수 있다. Vector similarity 만으로 해결하기 어려운 contradiction 과 temporal query 에 강하다. Graph maintenance 자체가 background compute 일수 있지만 foundation weight 를 바꾸지 않는다. 즉 external-memory STC 의 대표 medium 이다.

대안 3 — Recurrent/learned state

RNN, state-space model, Titans-like neural memory 는 긴 prefix 를 fixed-size 또는 learned state 로 압축한다. 매 token 의 context 재읽기를 줄일 수 있지만 state 가 어떤 prefix 에서 만들어졌는지 keying · sharing · cache invalidation 이 필요하다. 이 문제는 model architecture 와 serving infra 를 함께 요구한다.

대안 4 — Model editing

모델편집은 특정 factual association 을 localization 해 바꾼다. ROME 은 targeted edit, MEMIT 은 여러 memory 의 batch edit 가능성을 보였다. 그러나 bounded edit 성공은 lifelong sequence 에서 locality·retention·delete 가보장된다는 뜻이 아니다. “한 번 잘 고친다” 와 “영원히 안전하게 축적한다” 는다른 문제다.

대안 5 — Adapter 와 latent compilation

LoRA 는 weight delta 를 low-rank 로 제한해 trainable parameter 와 optimizer state 를 줄인다. Generative Adapter 처럼 context 를 한 번의 forward 로 parameter-like state 로 compile 하는 접근은 iterative gradient training 을피할 수 있다. 공통 관건은 compile cost 가 later reuse 로 상각되는지, version 과 tenant 별 delta 를 어떻게 관리하는지다.

대안 6 —Self-adaptation

Self-Adapting Language Models 계열은 model 이 adaptation data 와 update rule 을 스스로 생성한다. 이는 wake 와 sleep 사이를 잊지만, synthetic data quality 와 recursive error 가 핵심 risk 다. 좋은 data generation policy 를학습하는 것과 generated data 가 truth 를 보존하는 것은 별개다.

예와 반례

예 — 회사 규정 FAQ

규정은 출처·effective date·삭제가 중요하므로 temporal document store+RAG 가 우선이다. 질문 빈도가 높은 stable procedure 는 작은 domain adapter 로 promotion 할 수 있다. 매일 바뀌는 숫자까지 adapter 에 넣으면 stale weight 가 된다. Sleep job 은 모든 정보를 weight 로 옮기는 것이 아니라 promotion 후보를 평가한다.

예 — 개인 글쓰기 style

Style 은 하나의 원문 passage 보다 많은 episode 의 공통 pattern 이다. Retrieval 로 매번 예시를 넣을 수 있지만 token cost 가 반복된다. 충분한 consent 와 검증이 있다면 user adapter 가 효율적일 수 있다. 그러나 잘못학습된 style 은 모든 response 에 확산되므로 versioned rollback 이 필수다.

반례 — “Weight 면 retrieval 이 필요 없어 항상빠르다”

Adapter routing, loading, batching fragmentation, cache invalidation 까지포함하면 per-user weight 는 공짜가 아니다. 수백만 user 의 sparse delta 는 storage·metadata·activation 문제를 만든다. 또한 source citation 이필요하면 결국 external store 를 유지해야 한다.

반례 — “External memory 면안전하다”

Poisoned record 가 retrieval 되거나, summary 가 잘못 merge 되거나, delete 가 derived graph 까지 전파되지 않으면 external memory 도 위험하다. 외부라는사실은 observability 가능성을 높일 뿐, 올바른 policy 를 자동 제공하지않는다.

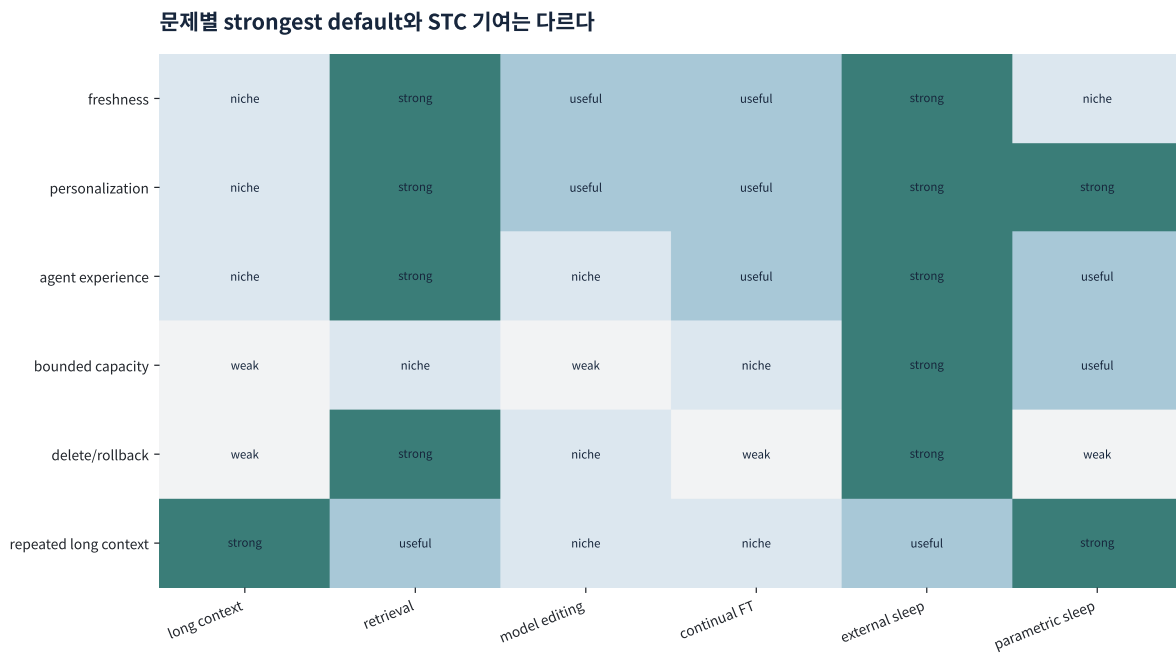


Figure 6: STC-F004. Problem-to-solution comparison matrix. 원 Study 의저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

그림 읽기

문제-대안 matrix 에서 하나의 열이 모든 행을 지배하지 않는다. RAG 는 provenance, adapter 는 repeated access, graph 는 temporal conflict, recurrent state 는 sequence compression 에서 강하다. 비교의 단위는 accuracy 하나가 아니라 update cost, read latency, interference, delete, rollback, reuse 다.

Hybrid architecture 는 external system-of-record 를 아래에 두고, 검증된 high-reuse knowledge 만 reversible adapter 나 cache 로 promotion 한다. 이그림에서 sleep cluster 는 source-of-truth 가 아니다. evidence 를 읽고 derived state 를 생성하는 compiler 다. 따라서 derived artifact 는 언제든지 원 evidence 에서 rebuild 가능해야 한다.

대안과 비교

접근	가장 잘 푸는 문제	reusable state	sleep 이 추가하는 가치	결정적 약점
long context	한 번의 깊은 이해	현재 context	사전 selec-tion·summary	매 요청 재독해
RAG/vector	factual lookup	document/in-dex	dedup·index rebuild	retrieval miss
temporal KG	변화 · 관계	graph	merge·super-sede	graph maintenance
recurrent state	prefix compression	hidden/neural state	snapshot·cache	state reuse/keying
editing	targeted correction	base weight	batch evaluation	lifelong locality
LoRA/adapter	stable repeated behavior	delta weight	promotion·ver-sion	fragmenta-tion/delete

접근	가장 잘 푸는 문제	reusable state	sleep 이 추가하는 가치	결정적 약점
latent compiler	fast parameterization	generated delta	batch compilation	fidelity·amortization

가장 합리적인 선택 순서는 “외부에 남겨도 되는가?” 부터 묻는 것이다. 예라면 external memory 가 governance 기준선이다. 이후 reuse 가 높고 stable 하며 latency 가치가 큰 subset 만 parametric state 로 올린다. 이순서를 뒤집으면 모든 transient fact 가 weight 에 들어가 capacity 와 deletion debt 를 만든다.

아직 모르는것

- Retrieval token 을 adapter 로 compile 할 때 정확히 몇 회의 reuse 에서 break-even 인지 workload 별 측정이 부족하다.
- Generative Adapter 같은 one-pass compilation 이 iterative fine-tuning 의 retention 과 locality 를 어디까지 따라가는지 대규모 evidence 가 없다.
- Recurrent state cache 의 cross-request reuse 율과 invalidation cost 가 KV prefix cache 와 공정하게 비교되지 않았다.
- Editing·LoRA·external memory 를 동일한 continual stream, delete request, poisoning attack 에서 비교한 benchmark 가 부족하다.
- Self-generated adaptation data 의 quality 를 external ground truth 없이 평가하는 방법이 미해결이다.

핵심

Sleep 은 “RAG 다음의 새 memory” 가 아니다. RAG·graph·adapter·state 를 어떤 cadence 와 evidence gate 로 유지할지 결정하는 lifecycle layer 다.

더 깊이 읽기

- 원 분석: 최신대안 frontier 와 선택 조건
- Training Background: 외부기억, LoRA, adapter, 모델편집, test-time training, 합성데이터
- Claim route: **STC-C026** · author_claim · confidence high · **STC-C027** · synthesis · confidence high · **STC-C028** · synthesis · confidence high · **STC-C029** · synthesis · confidence high · **STC-C030** · synthesis · confidence high · **STC-C031** · synthesis · confidence high

Hybrid: external system-of-record → selective reversible promotion

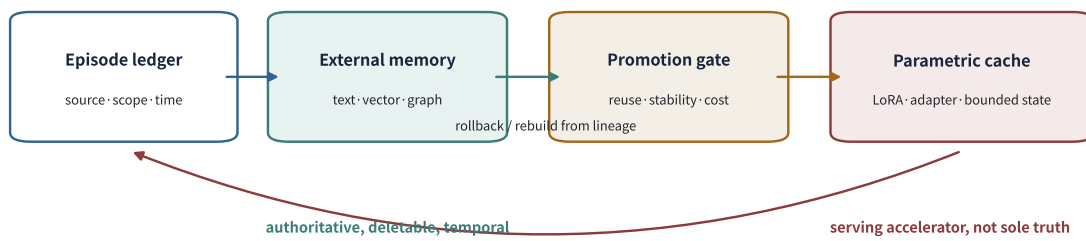


Figure 7: STC-F006. Hybrid system-of-record and selective-promotion architecture. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은원 Study 와 figure ledger 를 따른다.

E05 · Sleep phase 안에서 실제로 무엇을 계산하는가

이 권이 답할질문

“자는 동안 학습한다” 를 실제 연산 단계로 풀면 무엇이 남는가? Sleep phase 는 하나의 algorithm 이 아니라 data construction, selection, replay, distillation, adapter optimization, evaluation, state publication 을 엮은 pipeline 이다. 어느 단계를 쓰느냐에 따라 compute 와 memory traffic 이 완전히 달라진다.

이 권은 sleep mechanism taxonomy 를 따라 **STC-C015 · synthesis · confidence high**, **STC-C019 · author_claim · confidence high**, **STC-C023 · synthesis · confidence high**, **STC-C024 · synthesis · confidence high**, **STC-C035 · author_claim · confidence high** 을 연산 관점으로 설명한다.

먼저 세 문장

1. External-memory sleep 의 기본 loop 는 **episode 수집→추출→중복제거→충돌 해결→summary/graph/index 갱신→검증**이다. Gradient 가 없어도 generation 과 search 가 크다.
2. Parametric sleep 은 **replay data 구성→teacher/student 또는 task loss 계산→backward→optimizer→evaluation→adapter/weight version 배포**다. LoRA 로 weight write 를 줄여도 activation 과 data movement 는 남는다.
3. Dreaming 이나 synthetic augmentation 은 data 부족을 줄이지만, 자기 output 을 다시 학습하는 recursive loop 이므로 real evidence anchor 와 quality gate 가 필요하다.

직관

Mechanism A — Replay

과거 data 또는 그것을 흉내 낸 sample 을 새 data 와 섞어 학습한다. 원 sample 을 보존하면 fidelity 가 높지만 storage·privacy cost 가 있다. 합성데이터로 replay 하면 저장량을 줄일 수 있으나 rare case 와 failure mode 를 generator 가 놓칠 수 있다. Replay 는 “기억을 다시 보여 주는 data plane” 이다.

Mechanism B — Distillation

Teacher 의 output distribution 이나 behavior 를 student state 로 옮긴다. 지식증류는 raw label 보다 richer signal 을 줄 수 있고, large context/fast memory 를 compact adapter 로 옮기는 데 쓰인다. 하지만 compression 은 무엇을 버릴지 결정한다. Teacher 가 틀렸거나 student capacity 가 부족하면 오류도 함께 압축된다.

Mechanism C — Data augmentation 과 dreaming

Model 이 스스로 candidate experience 를 만들고 evaluator 가 useful sample 을 고른다. 데이터 증강의 장점은 실제 log 에 없던 corner case 를 탐색할 수 있다는 점이다. Reward 나 downstream improvement 로 sample 을 고르면 보상설계가 data distribution 을 지배한다. “그럴듯함” 과 “학습 후 utility” 는 다르다.

Mechanism D — Adapter update

LoRA 는 full weight 대신 low-rank delta 를 학습한다. Trainable parameter 와 optimizer state 가 줄고 version 단위가 작아진다. 그러나 forward activation, backward gradient, base weight read 는 여전히 필요하다. 또 user 별 adapter 가 많아지면 batch 가 갈라지고 routing metadata 가 커진다.

Mechanism E — Consolidation 과 compaction

여러 fast memory 를 slow memory 로 옮기거나, 여러 episode 를 summary/graph 로 압축한다. 공고화는중요한 pattern 을 장기 state 로 promotion 하는 과정이다. External version 은 merge·supersede·index rebuild 이고, parametric version 은 distillation·fine-tuning·expert expansion 이다.

Mechanism F — RL-style optimization

Dream candidate 나 adapter policy 가 future task score 를 높이면 reward 를준다. 정책기울기를 쓸 수도 있고 best-of-N selection 처럼 gradient 없는 최적화를 쓸수도 있다. 핵심은 reward evaluation 이 sleep cost 의 일부이며, reward hacking 과 distribution shift 를 막아야 한다는 점이다.

시스템 모델링 관점

Sleep job 의 FLOP 만 계산하면 불완전하다. Episode read bytes, vector/graph random write, teacher rollout token, activation checkpoint, optimizer state, adapter snapshot, validation inference, failed version 폐기까지포함해야 한다.

예와 반례

External synthesis 한 cycle

1. 지난 24 시간의 consented episode 를 immutable log 에서 읽는다.
2. 후보 fact·preference·strategy 를 structured record 로 추출한다.
3. 기존 memory 와 semantic·temporal conflict 를 찾는다.
4. admit/merge/supersede/reject 를 결정하고 provenance edge 를 남긴다.
5. vector index 와 temporal graph 를 incremental update 한다.
6. held-out query 로 retrieval precision 과 contradiction 을 검사한다.
7. 통과한 generation 만 active pointer 로 전환한다.

이 cycle 은 foundation training 보다 가볍지만 embedding, LLM extraction, graph traversal, random write 가 섞인 heterogeneous workload 다.

Parametric adapter 한 cycle

1. External system-of-record 에서 high-reuse-low-conflict record 를선택한다.
2. Positive example, counterexample, deletion test, old-capability replay 를 dataset 으로 만든다.
3. Teacher output 과 ground truth 를 섞어 task/distillation loss 를계산한다.
4. Adapter 만 backward·optimizer update 한다.

두 개의 독립 축: update timing × memory medium

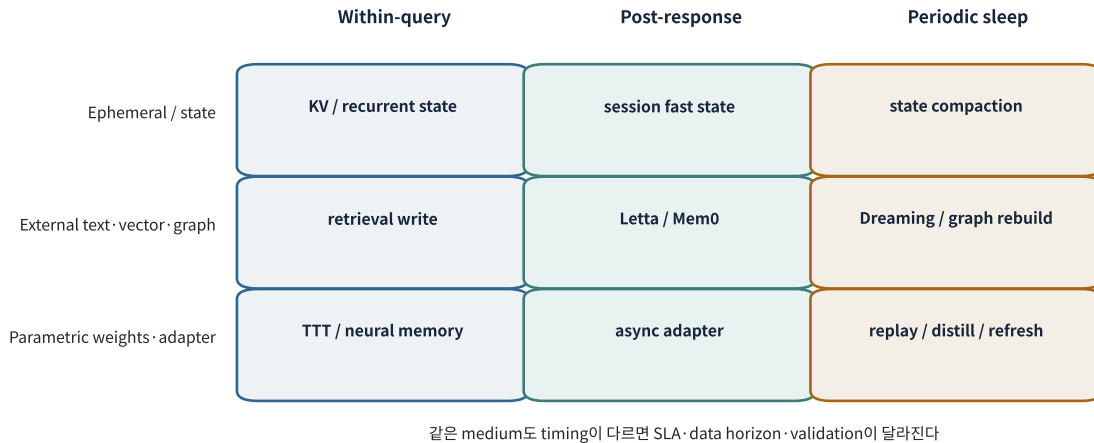


Figure 8: STC-F002. Update timing by memory-medium taxonomy. 원 Study 의저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

5. New fact, old skill, locality, privacy, adversarial set 을 평가한다.
6. Threshold 를 통과하면 versioned adapter 를 publish 하고 canary traffic 에붙인다.
7. Regression 이면 pointer 를 이전 generation 으로 되돌린다.

반례 — Training loss 가 낮으니 consolidation 성공

Train loss 는 dataset 을 얼마나 잘 맞췄는지만 말한다. Old capability 가사라졌는지, unrelated answer 가 변했는지, delete request 가 반영되는지, future query latency 가 줄었는지는 별도 평가다. Sleep 의 output 은 model artifact 이므로 release gate 가 필요하다.

반례 — Dream sample 수를 늘리면 계속좋아짐

Synthetic sample 이 같은 generator bias 를 반복하면 quantity 가 diversity 를보장하지 않는다. Real data anchor 를 잃으면 distribution tail 이 줄어드는 model collapse 가 가능하다. Generated replay 는 free data 가 아니라 provenance 가 약한 derived data 다.

그림 읽기

Mechanism 을 memory medium 위에 배치해 보라. Replay·distillation 은 adapter 와 weight 쪽에, dedup·graph merge 는 external memory 쪽에, recurrent update 는 learned state 쪽에 놓인다. 동일 sleep scheduler 가서로 다른 kernel 과 storage pattern 을 부른다. 그래서 “sleep cluster” 는단일 GPU cluster 라기보다 GPU training pool, CPU/near-data preprocessing, graph/index tier 를 연결한 fabric 이다.

Failure-mode 그림은 pipeline 의 어느 지점에 control 을 넣어야 하는지 보여준다. Ingest 에는 consent-poison filtering, transform 에는 provenance·quality score, train 에는 replay·regularization, publish 에는 regression·canary, serve 에는 observability, delete 에는 lineage-driven rebuild 가 필요하다. Control 이 없는 sleep 은 오류를 durable 하게 만드는자동화다.

Failure mode마다 lifecycle control을 붙인다

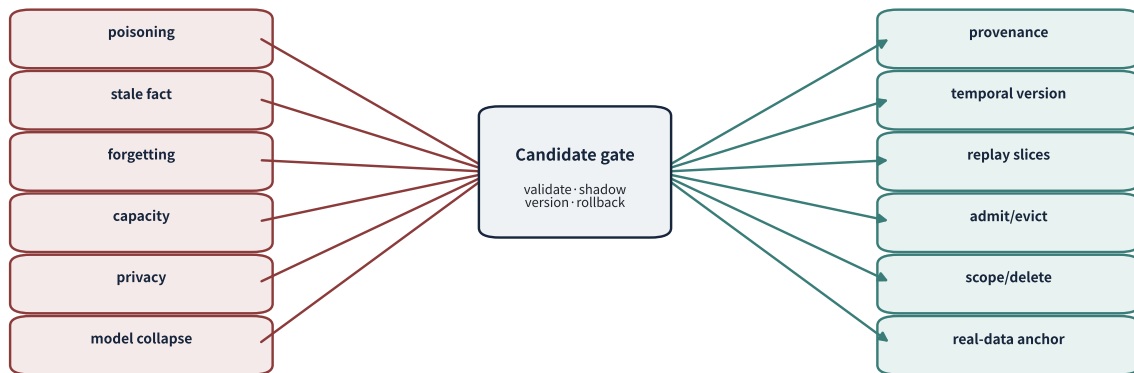


Figure 9: STC-F013. Failure modes and lifecycle controls. 원 Study 의 저자작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

대안과 비교

mechanism	주된 연산	주된 state traffic	장점	핵심 risk
summary/d-edup graph update	generation, similarity extraction, traversal	episode read, record write random	해석 · 삭제 쉬움 시간 · 관계표현	lossy merge
replay SFT	forward/back- ward	activation, optimizer	강한 retention baseline	write amplification storage/privacy
distillation	teacher+stu- dent forward, backward	logits/activa- tion	compact behavior transfer	teacher error
LoRA	base forward+delta backward	base read, small weight write	작은 artifact	adapter explosion
dreaming/RL	rollout, reward, update	generated data, checkpoints	data exploration	recursive bias

Mechanism 선택은 “얼마나 AI 다운가” 가 아니라 memory utility 와 system cost 의 곱으로 결정해야 한다. Stable factual state 는 graph/record 가, repeated behavior 는 adapter 가, rare raw evidence 는 cold episode store 가 더 적합할 수 있다.

아직 모르는것

- Sleep cycle 별 end-to-end energy·wall-clock·data-movement breakdown 이 공개된 연구가 드물다.
- Teacher rollout, reward model, LoRA SFT 중 어느 향이 실제 utility 를 만드는지 ablation 이 충분하지 않다.
- Generated replay 가 rare tail 을 얼마나 보존하는지 장기 sequence 에서 측정되지 않았다.
- Multi-rate neural memory update 가 prefix cache reuse 와 함께 실제 serving throughput 을 높이는지 실증이 부족하다.

- Online consolidation 과 scheduled sleep 사이에서 optimal split 을 학습하는 scheduler 가 없다.

핵심

Sleep 은 “한 번 더 fine-tuning” 이 아니다. Evidence 를 data 로 만들고, state 를 갱신하고, 검증해 배포하는 작은 ML lifecycle 전체다.

더 깊이 읽기

- 원 분석: replay·distill·RL·external compaction mechanism
- Training Background: 증류, 데이터증강, 합성데이터, LoRA, 보상, 정책기울기, 공고화
- Claim route: **STC-C015** · synthesis · confidence high · **STC-C019** · author_claim · confidence high · **STC-C023** · synthesis · confidence high · **STC-C024** · synthesis · confidence high · **STC-C035** · author_claim · confidence high

E06 · 강한 대안과 정면 비교하면 무엇이남는가

이 권이 답할질문

Sleep-time compute 가 promising 하다는 판단은 약한 baseline 과의 accuracy 비교로는 설득력이 없다. “그 문제를 지금 가장 잘 푸는 대안” 과 같은 compute·memory·governance budget 에서 비교해야 한다. 이 권은 replay, EWC, GEM, parameter isolation, RAG, model editing, rate-distortion compaction 을 sleep 과 같은 표에 놓는다.

기준은 strongest-alternative comparison 이며, 근거는 **STC-C020** · author_claim · confidence high, **STC-C021** · author_claim · confidence high, **STC-C026** · author_claim · confidence high, **STC-C034** · synthesis · confidence high, **STC-C040** · synthesis · confidence medium 이다.

먼저 세 문장

1. 망각만 줄이는 문제라면 replay·EWC·GEM·parameter isolation 이 이미 강한 baseline 이다. Sleep 은 이들보다 retention 뿐 아니라 lifecycle cost 와 governance 에서 더 나아야 한다.
2. 사실 기억은 RAG/temporal graph 가 출처 · 수정 · 삭제 면에서 우세하다. Parametric sleep 은 repeated access 나 latent skill 처럼 weight 가 실제로유리한 subset 에 한정해야 한다.
3. 현재 evidence 를 종합하면 가장 강한 system hypothesis 는 **external system-of-record + selective reversible promotion** 의 hybrid 다.

직관

EWC — 중요한 parameter 를 덜움직인다

EWC 는 old task 에 중요하다고 본 parameter 의 이동에 큰 penalty 를 준다. 저장해야할 것은 과거 parameter 와 importance estimate 다. 장점은 raw replay 가없어도 된다는 점이고, 약점은 task 가 많아질수록 penalty 가 새 학습을 막고 importance approximation 이 부정확할 수 있다는 점이다.

GEM — 과거 loss 를 악화시키지 않는 방향으로움직인다

GEM 은 replay buffer 의 old example 에 대해 loss 가 증가하지 않도록 new gradient 를 projection 한다. Local constraint 가 명확하지만 buffer·gradient comparison cost 가 있다. 모든 future behavior 의 불변을 보장하는 것은 아니다.

Parameter isolation — 충돌할 공간을분리한다

Adapter, expert, mask, progressive network 는 task 나 user 별 parameter 를분리한다. 파라미터격리는 interference 를 줄이지만 capacity 가 단조 증가하고 routing·batch fragmentation 이 생긴다. Dynamic expansion 은 “잊지 않기” 를 “계속늘리기” 로 바꾼다.

Rate - distortion compaction — 무엇을 버릴지수치화한다

율-왜곡관점은 memory byte 라는 rate 와 answer utility 손실이라는 distortion 의 tradeoff 를 명시한다. 모든 memory 를 보존할 수 없으므로 어떤 detail 을압축할지 결정한다. 이론적 framing 은 강하지만 agent 의 실제 utility 와 distortion 을 어떻게 정의할지는 열린 문제다.

RAG — Mutable truth 를 밖에둔다

External store 는 audit·delete·freshness 의 기준선이다. Weakness 는 retrieval miss 와 token cost 다. Sleep synthesis 로 index 와 summary 를 개선할 수 있지만 source-of-record 를 weight 로 대체할 이유는 없다.

예와 반례

공정한 comparison protocol

동일한 1 년짜리 user stream 을 가정한다. 각 method 에 같은 total compute, 같은 retained byte, 같은 evaluation query, 같은 delete request 를 준다. 다음을 함께 쟀다.

- 새 정보 적응: current utility
- 옛 정보 보존: backward retention
- unrelated behavior 변화: locality
- 원 evidence 연결: provenance coverage
- 삭제 후 잔존: delete leakage
- update/serve cost: joule, byte, latency
- rollback: 이전 generation 복구 시간

이 protocol 에서 accuracy 만 높은 method 는 승자가 아니다. 예를 들어 adapter 가 빠르게 답해도 delete leakage 가 크면 regulated domain 에서 탈락할 수 있다.

예 — Stable skill promotion

도메인별 수백 개 procedure 를 external store 에서 제공하다가, query frequency 가 높고 내용이 3 개월 이상 안정된 subset 만 adapter 로 distill 한다. 원 record 와 adapter version lineage 를 유지하고, 변경 시 adapter 를 rebuild 한다. 이 hybrid 는 RAG 의 governance 와 adapter 의 low-latency access 를 결합한다.

반례 — 모든 user 에게 별도 full model

Parameter isolation 을 극단적으로 쓰면 interference 는 줄지만 model count 와 memory 가 user 수에 비례한다. Hardware memory 를 늘려도 scheduler·batching·cold-start 가 병목이 된다. 중요한 것은 parameter 수가 아니라 active working set 과 reuse distribution 이다.

반례 — 무제한 raw replay

Raw replay 는 강한 retention baseline 이지만 privacy 와 storage 를 무시할 수 없다. Consent 철회가 일어나면 training example 과 그것에서 파생된 state 를 찾아 제거해야 한다. Replay buffer 가 system-of-record 가 되면 data governance 비용이 model cost 보다 커질 수 있다.

그림 읽기

이 matrix 는 method 를 단일 ranking 으로 만들지 않는다. 각 problem row 마다가 가장 강한 baseline 이 다르다. Sleep 은 timing column 을 더해 batch evaluation 과 cross-episode consolidation 을

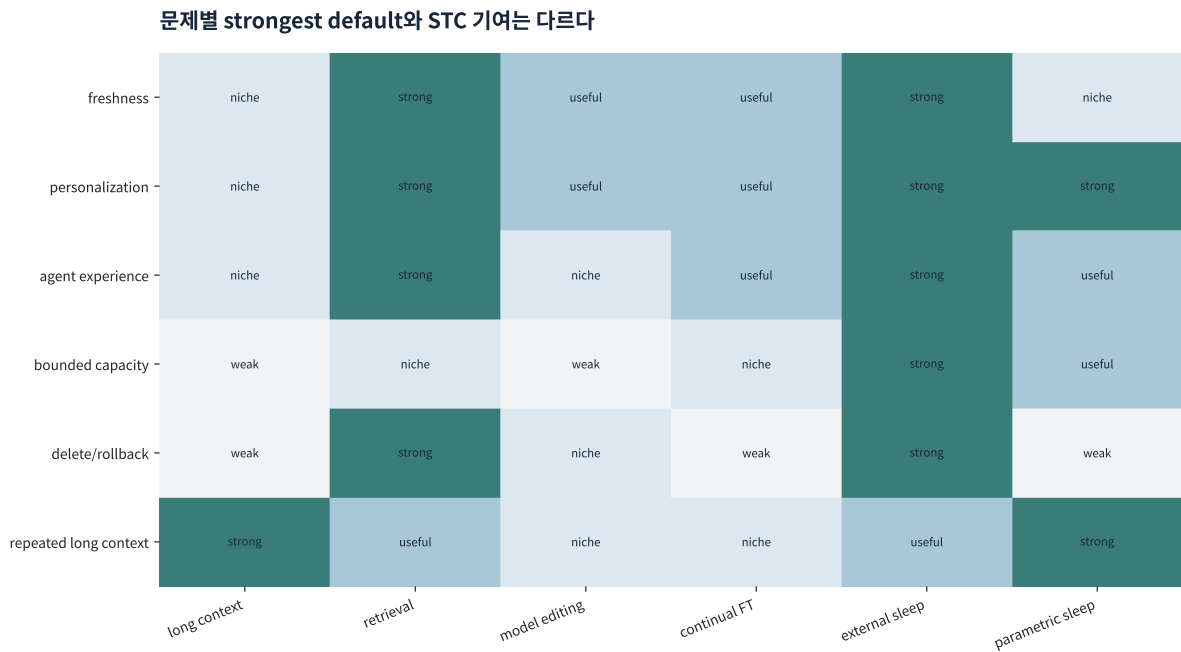


Figure 10: STC-F004. Problem-to-solution comparison matrix. 원 Study 의저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

Hybrid: external system-of-record → selective reversible promotion

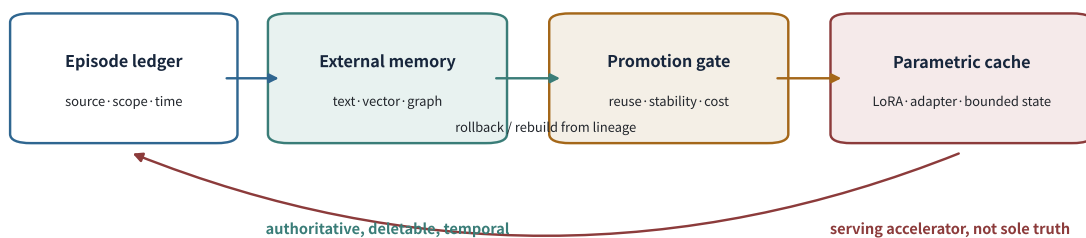


Figure 11: STC-F006. Hybrid system-of-record and selective-promotion architecture. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

제공하지만, medium 의 약점을 없애지 않는다. Weight sleep 은 여전히 delete 가 어렵고, external sleep 은 여전히 retrieval 이 필요하다.

Hybrid 에서 admission gate 는 “이 기억이 중요해 보이는가” 만 보지 않는다. Reuse, stability, confidence, privacy, reversibility 를 함께 본다. Promotion 은 one-way migration 이 아니라 source-of-record 에서 다시 생성가능한 cache/compiled artifact 여야 한다. 이 원칙이 weight 를 authoritative memory 로 두는 설계와 가장 큰 차이이다.

대안과 비교

방법	retention	plasticity	capacity growth	prove-nance/delete	serving cost
replay	높음, data quality 의존	높음	buffer 증가	raw data 는 추적가능	sleep train 큼
EWC	중간	task 증가 시 낮아짐	importance state 증가	weight source 불 명확	update 계산 추가
GEM	local constraint 강함	projection 범 위 내	buffer 증가	example lineage 가능	gradient projection
adapter isolation	충돌 적음	새 slot 에 높음	단조 증가	version 단위 rollback	routing fragmentation
RAG/graph	원문 보존	즉시 update	record/index 증가	가장 강함	retrieval+tokens
hybrid sleep	선택에 따라	선택에 따라	tier 별 관리	external anchor 유지	orchestration 복잡

Strongest alternative 관점에서 sleep 이 우월해질 조건은 세 가지다. 첫째, 여러 episode 를 batch 로 봐야만 얻는 pattern 이 있어야 한다. 둘째, 그 pattern 이 미래에 충분히 재사용되어 sleep cost 를 상각해야 한다. 셋째, derived state 를 audit·rollback·delete 할 수 있어야 한다.

아직 모르는것

- Hybrid promotion gate 를 어떤 signal 로 학습해야 false promotion 을 줄이는지 모른다.
- Rate – distortion 의 distortion 을 answer accuracy, preference fidelity, safety 중 어떻게 합칠지 합의가 없다.
- Adapter isolation 의 long-tail user 분포에서 active set, cache hit, batching loss 측정이 부족하다.
- EWC/GEM/replay 와 modern external-memory product 를 같은 long-lived agent benchmark 에서 비교한 연구가 없다.
- Parametric memory 를 external record 에서 완전히 rebuild 할 수 있는 reproducibility standard 가 없다.

핵심

Sleep 의 strongest case 는 “모든 기억을 weight 로 옮긴다” 가 아니라 “외부 truth 를 유지하면서 반복 가치가 검증된 일부만 되돌릴 수 있게 compile 한다” 다.

더 깊이 읽기

- 원 분석: strongest alternatives 와 hybrid verdict
- Training Background: EWC, GEM, 파라미터격리, 울-왜곡, 기억입장, 기억퇴거
- Claim route: **STC-C020** · author_claim · confidence high · **STC-C021** · author_claim · confidence high · **STC-C026** · author_claim · confidence high · **STC-C034** · synthesis · confidence high · **STC-C040** · synthesis · confidence medium

E07 · 논문 · 제품 · negative evidence 는 어디까지왔는가

이 권이 답할질문

Sleep-time compute 가 학계 · 업계의 실제 방향인가, 몇 편의 논문을 큰 trend 로 읽은 것인가? 답하려면 paper count 가 아니라 evidence layer 를 나눠야 한다: biological precursor, continual-learning algorithm, explicit parametric sleep, production external-memory synthesis, benchmark, negative evidence. 2026-08-05 freeze 에서 가장 강한 production signal 은 external-memory background synthesis 이고, broad per-user foundation-weight sleep 은 아직 연구 단계다.

이 권은 evidence trajectory 의 73 개 고정 source audit 를 설명한다. 중심 claim 은 **STC-C001** · direct_fact · confidence high, **STC-C002** · direct_fact · confidence high, **STC-C003** · direct_fact · confidence high, **STC-C007** · direct_fact · confidence high, **STC-C008** · direct_fact · confidence high, **STC-C009** · direct_fact · confidence high, **STC-C047** · direct_fact · confidence high 이다.

먼저 세 문장

1. 논문은 “가능성”, 제품 문서는 “운영 의도”, 공개 code 는 “구현”, 장기 telemetry 는 “경제성” 을 보여 준다. 이 네 evidence 를 섞으면 성숙도를과장한다.
2. OpenAI Dreaming, Mem0 Dream, Letta sleeptime 은 응답 밖에서 외부 memory 를 합성 · 정리하는 lifecycle 의 production signal 이다. 공개 자료는이것을 per-user foundation-weight training 이라고 말하지 않는다.
3. Parametric sleep 은 Google 계열의 명시적 연구를 포함해 유망한 증거가있지만, fleet-scale retention·rollback·delete·security 가 동시에 검증된공개 system 은 확인되지 않았다.

직관

Evidence ladder

1. **Analogy** — 뇌의 수면처럼 phase 를 나누면 좋을 수 있다.
2. **Toy/mechanistic experiment** — 제한된 task 에서 forgetting 이줄었다.
3. **Modern model experiment** — billion-scale checkpoint 에서 improvement 가 나왔다.
4. **Open implementation** — code 와 config 로 mechanism 을 재현할 수있다.
5. **Product behavior** — 실제 사용자 memory 에 background job 이적용된다.
6. **Longitudinal operations** — 수개월 retention, cost, rollback, abuse 가 측정된다.

현재 external-memory sleep 은 4 - 5 단계의 신호가 있고, parametric sleep 은주로 2 - 3 단계다. 6 단계의 공개 evidence 는 부족하다.

날짜 freeze 가 중요한이유

이 분야는 paper 와 product 가 빠르게 바뀐다. “최신” 이라는 표현은 반드시 cutoff 와 version 을 가져야 한다. 본 연구는 2026-08-05 에 source URL, release, repository tag, paper version 을 고정했다. 출처추적성은 memory 뿐 아니라 research claim 에도 필요하다.

Positive evidence 와 negative evidence

Positive result 만 모으면 trend 가 언제나 강해 보인다. 반대로 capacity, model collapse, memory poisoning, serving reuse 같은 실패 근거를 함께보면 조건부 verdict 가 된다. DANN/SSRN record 는 method 와 result 가 독립검증에 충분하지 않아 load-bearing evidence 에서 제외했다. 이것은 “틀렸다” 는 판정이 아니라 evidence hygiene 다.

주의

회사 이름이 paper 에 등장한다는 사실과 회사의 production roadmap 은 다르다. 공개 자료가 말한 medium 과 scope 를 넘어 추론하지 않는다.

예와 반례

OpenAI Dreaming

공개 설명은 background synthesis 가 user memory 를 연결하고 더 유용한 derived memory 를 만든다는 product signal 이다. 중요한 한정은 external user memory 라는 점이다. 공개 설명만으로 per-user foundation weight 가 변경된다고 주장할 수 없다. 따라서 industry adoption evidence 에는 포함하지만 parametric fleet evidence 에는 포함하지 않는다.

Mem0 Dream

Scheduled background synthesis 와 provenance-preserving additive output 이 문서 ·tagged code 에서 확인된다. Merge 와 supersede 는 별도 ingest operation 이다. 이 구분은 sleep output 이 원 record 를 파괴하는 in-place rewrite 가 아니라 derived layer 일 수 있음을 보여 준다.

Letta sleeptime

Tagged group 은 response 후 asynchronous agent 가 durable memory 를 수정하도록 schedule 한다. 이 역시 “sleep” 이라는 이름을 실제 orchestration 에 연결한다. 하지만 동일한 label 을 쓰는 모든 implementation 이 같은 quality gate 나 evidence semantics 를 갖는 것은 아니다.

반례 — Benchmark 하나의 SOTA

특정 task 에서 sleep method 가 baseline 을 이겼더라도 다른 retention stream, deletion, poisoning, cost 에서 우월하다는 뜻은 아니다. 검증세트와 테스트세트를 반복 tuning 에 혼용하면 성능도 과대평가된다.

반례 — 많은 citation = mainstream

Research attention 은 adoption 가능성을 높이지만 infra deployment 를 보장하지 않는다. Training job, user state version, compliance delete 가 추가되는 system 은 작은 accuracy gain 만으로 도입되지 않는다.

1995–2026: 이름보다 먼저 존재한 계보



Figure 12: STC-F003. 1995-2026 evidence chronology. 원 Study 의 저자 작성합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

그림 읽기

Timeline에서 점의 개수보다 layer 이동을 본다. 초기에는 memory theory 와 forgetting algorithm 이 중심이고, 2020 년대에는 LLM external memory, neural memory, background product 가 접친다. “sleep” 이라는 단어는 늦게 등장하지만 offline consolidation 기능은 오래전부터 있었다.

Landscape 는 academia 와 industry 를 medium 별로 나눈다. Industry 의 밀도는 external memory·synthesis 쪽이 높고, academia 는 parametric update·continual learning·capacity theory 쪽이 깊다. 두 집단의 교차점이 hybrid system 과 benchmark 다. 이것이 “lifecycle 은 주류가 될 수 있지만 parametric sleep 이 곧 주류라는 뜻은 아니다” 라는 판단의 근거다.

대안과 비교

evidence 종류	말할 수 있는 것	말할 수 없는 것	필요한 다음 증거
biological model	phase separation mechanism	LLM fleet economics	modern-model ablation
continual-learning paper	retention under task stream	agent governance	long-lived agent benchmark
explicit sleep paper	method-level gain	production reliability	code, multi-cycle replication
product docs	deployed feature intent	internal weight update	architecture/telemetry disclosure
repository tag	implementation behavior	user-scale outcomes	longitudinal metrics
benchmark	common measurement	best architecture	multiple systems + cost

출판게이트는 claim 의 강도를 evidence ladder 에 맞춘다. “보였다” , “제안했다” , “배포했다” , “장기적으로 검증했다” 를 서로 바꾸지 않는 것이 핵심이다.

아직 모르는것

- Product sleep 이 실제 user utility, retention, cost 에 미치는 longitudinal telemetry 가 공개되지 않았다.

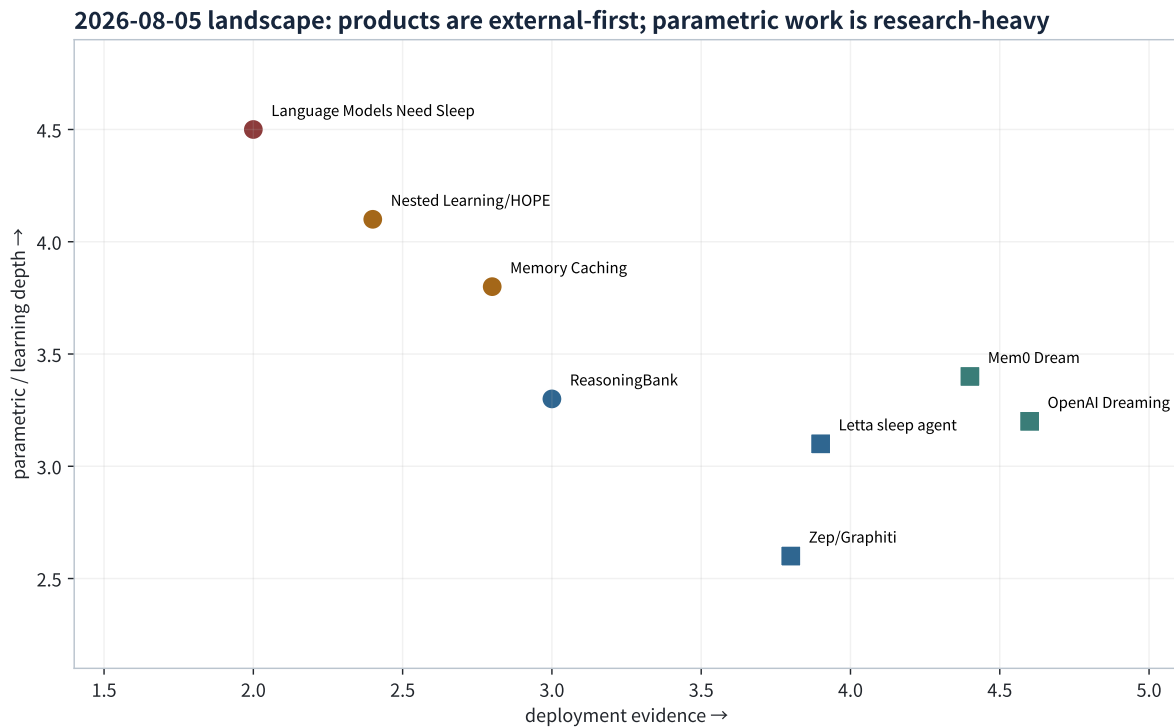


Figure 13: STC-F014. Industry and academia landscape at the source freeze. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

- Parametric sleep 의 수백 cycle stability 와 adapter generation 수가보고되지 않았다.
- External synthesis 의 잘못된 memory merge 율과 human correction cost 가정량화되지 않았다.
- 회사별 “dreaming/synthesis/reflection” 이 서로 다른 기능인데 공통 benchmark 가 없다.
- Search saturation audit 의 12 cluster 중 active frontier 와 measurement gap 은 future update 가 필요하다.

핵심

2026 년의 정직한 결론은 “background memory lifecycle 은 이미 제품화신호가 강하다. Broad parametric sleep 은 promising research 지만 mainstream production evidence 는 아직 부족하다” 다.

더 깊이 읽기

- 원 분석: date-frozen evidence trajectory 와 industry/academia 구분
- Training Background: 검증세트, 테스트세트, 출판게이트, 출처추적성, 관측가능성
- Claim route: **STC-C001** · direct_fact · confidence high · **STC-C002** · direct_fact · confidence high · **STC-C003** · direct_fact · confidence high · **STC-C007** · direct_fact · confidence high · **STC-C008** · direct_fact · confidence high · **STC-C009** · direct_fact · confidence high · **STC-C047** · direct_fact · confidence high

E08 · 정말 promising 하며 주류가 될까

이 권이 답할질문

Sleep-time compute 이라는 새 scaling 축은 실제로 promising 한가? 학계 · 업계가 이 방향으로 발전할까? 이 질문에는 단일 yes/no 보다 어느 **variant** 가 어떤 문제에서 주류가 되는가가 더 정확하다. Background synthesis 와 external consolidation 은 이미 practical value 가보이지만, user 별 foundation-weight sleep 은 증거 · 안전 · economics 가 부족하다.

이 권은 promisingness 판정과 mainstream scenario 를 연결한다. 근거는 **STC-C038** · synthesis · confidence high, **STC-C039** · synthesis · confidence medium, **STC-C040** · synthesis · confidence medium, **STC-C042** · synthesis · confidence high 다.

먼저 세 문장

1. **Lifecycle** 은 **promising** 하다. 응답 밖에서 memory 를 정리 · 검증 · 압축하는 요구는 long-lived agent 가 늘수록 커진다.
2. **Medium** 별 전망은 다르다. External-memory sleep 은 provenance 와 update 가 쉬워 먼저 확산될 가능성이 높고, broad parametric sleep 은 selective reversible adapter 형태가 더 현실적이다.
3. “Sleep-time compute” 라는 용어가 주류가 되지 않아도 dreaming, reflection, memory synthesis, self-evolution, background learning 이라는 기능은 주류가 될 수 있다.

직관

Promisingness 를 네 조건으로 본다

1. **Problem pressure** — 현재 방법으로 남는 큰 문제가 있는가?
2. **Unique leverage** — 응답 후 batch 가 그 문제에 특별한 이점을주는가?
3. **Evidence maturity** — modern model 과 product 에서 재현되는가?
4. **Operational viability** — cost, rollback, delete, security 를 감당할 수 있는가?

External-memory sleep 은 1·2·4 에서 비교적 강하고 3 도 product signal 이 있다. Parametric sleep 은 1·2 의 잠재력은 크지만 3·4 가 약하다. 그래서 전체 verdict 는 “조건부 promising” 이다.

왜 수요가 생기는가

Agent 가 하루가 아니라 수개월 일하면 context 를 계속 늘릴 수 없다. Preference, project state, entity relation, reusable strategy 를 durable 하게 만들고 stale record 를 정리해야 한다. 이 일은 single request 중 처리하기 어렵고 여러 episode 를 비교해야 하므로 background phase 가 자연스럽다.

왜 full-weight personalization 이 바로 오지 않는가

모수적 기억은 inference access 가 빠르고 behavior 를 직접 바꿀 수 있다. 그러나 source attribution, user delete, poisoning containment, multi-tenant batching, version explosion 이 어렵다. Base model 을 user 마다 바꾸면 shared serving economics 가 깨진다. 작은 서버 산출물인 adapter 나 compiled cache 가 먼저 현실적이다.

왜 external-only 도 최종 답은 아닐 수있는가

외부기억은 audit 에 강하지만 every-query retrieval, context token, miss, prompt injection 비용이 있다. 매우 자주 재사용되는 stable pattern 은 parametric/latent cache 로 compile 하면 이득일 수 있다. 따라서 medium-term mainstream 은 hybrid 다.

예와 반례

Scenario A — External synthesis 가 기본 기능이 됨

Agent platform 은 episode log, user memory, temporal graph 를 유지하고 idle time 에 dedup·summary·conflict resolution 을 수행한다. Output 은 원 record 와 lineage 로 연결되고 사용자가 inspect·correct·delete 할 수 있다. 이 scenario 는 기술·규제 양쪽에서 가장 가까운 경로다.

Scenario B — Domain adapter promotion

기업 내부에서 반복되는 stable procedure 를 shared domain adapter 로 nightly/weekly compile 한다. Adapter 는 버전관리되고 canary 평가를 통과해야 하며, external knowledge base 가 authoritative source 로 남는다. User 별보다 domain 별 reuse 가 높아 batching economics 도 좋다.

Scenario C — On-device private sleep

개인 data 가 cloud 로 나가기 어렵고 device 가 idle/charging 일 때 local summary 나 small adapter 를 갱신한다. Privacy 는 좋아지지만 thermal, battery, write endurance, rollback UI 가 제약이다. 특정 workload 에는 유망하지만 universal default 는 아니다.

반례 — “Hyperscaler 가 paper 를 냈으니 곧 전 모델에 적용”

Research paper 는 option value 를 보여 준다. Production adoption 에는 inference architecture, abuse control, SRE, compliance 가 추가된다. 특히 user-specific weights 는 base-model sharing 과 cache locality 를 해칠 수 있어 경제성 검증이 필요하다.

반례 — “External product 가 있으니 scaling law 도 증명됨”

Feature 가 존재하는 것과 sleep FLOP 을 늘리면 utility 가 예측 가능하게 증가한다는 scaling law 는 다르다. 현재는 accounting identity 와 break-even hypothesis 가 있을 뿐 universal empirical law 는 없다.

그림 읽기

Conditional verdict 그림은 green/yellow/red 조건을 나눈다. Cross-episode synthesis, index maintenance, selective adapter 는 green/yellow 에 가깝다. Broad autonomous weight rewrite 는 evidence·governance 가 약해 red 에 가깝다. 그림의 목적은 기술을 찬반으로 나누는 것이 아니라 **promotion** 범위를 단계적으로 제한하는 것이다.

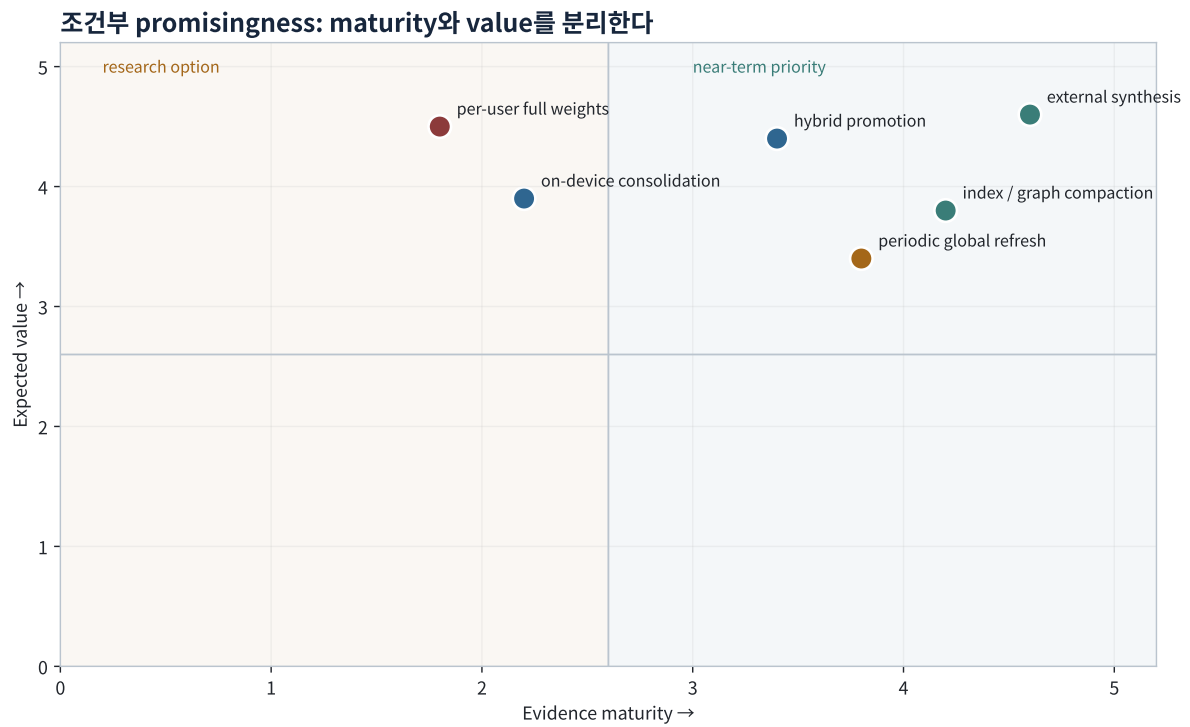


Figure 14: STC-F005. Conditional promisingness verdict. 원 Study 의 저자작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

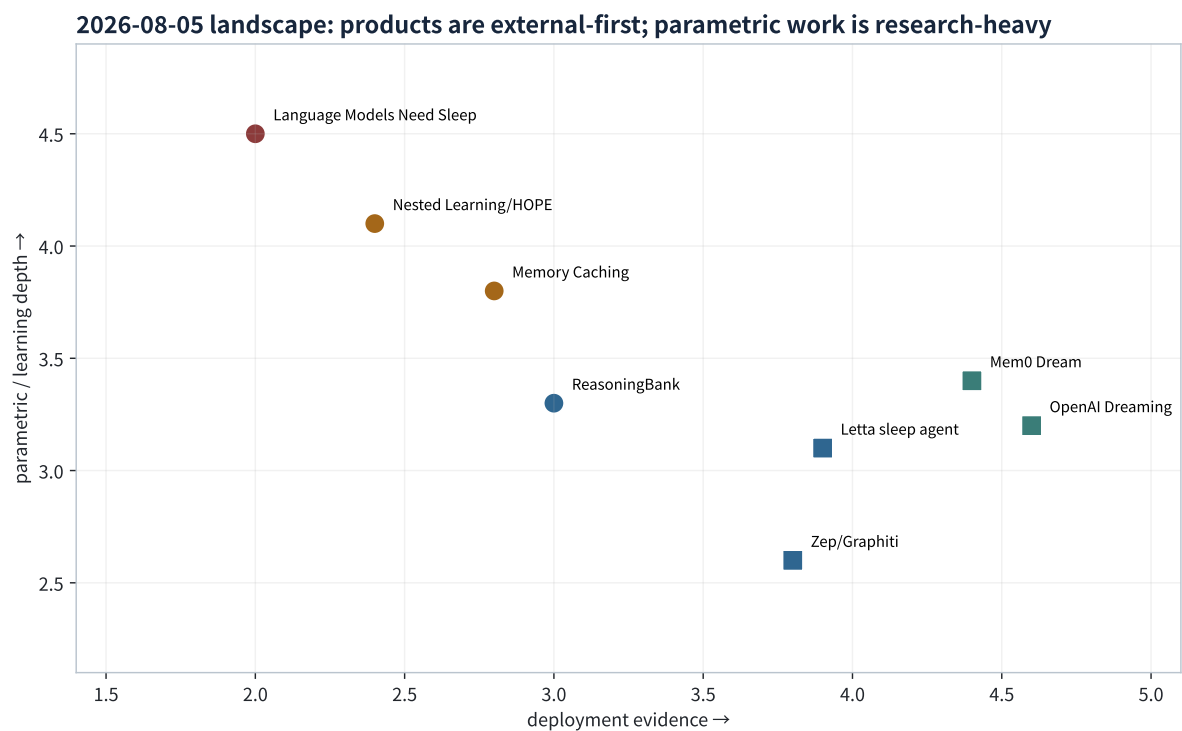


Figure 15: STC-F014. Industry and academia landscape at the source freeze. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

Academia 는 method frontier 를, industry 는 external-memory lifecycle 을 밀고 있다. 두 방향이 만나는 곳은 benchmark, hybrid architecture, reversible artifact, memory infra 다. Memory device 회사의 기회도 이교차점에 있다: 모든 weight training 을 가속하는 것보다 derived state 의 저장 · 이동 · version 을 최적화하는 것.

대안과 비교

전망	2-3 년 가능성	필요한 조건	주류화 장벽
external synthesis	높음	provenance, correction UX	quality drift, poisoning
temporal graph maintenance	중상	schema/temporal query	extraction error, write cost
shared domain adapter sleep	중간	reuse, canary, rollback	update pipeline, fragmentation
user adapter sleep	중하	consent, isolation, cache	millions of versions
foundation-weight sleep	낮음/연구	robust continual learning	delete, safety, fleet cost
learned cadence	연구 frontier	utility telemetry	delayed credit assignment

여기서 “주류” 는 모든 model 이 같은 implementation 을 쓴다는 뜻이 아니다. Database 에 backup·compaction 이 기본 기능이듯, agent platform 에 background memory lifecycle 이 기본 layer 가 되는 것을 뜻한다.

아직 모르는것

- User 가 background-derived memory 를 얼마나 신뢰하고 correction 하는지 product telemetry 가 부족하다.
- Sleep job 이 만든 utility 를 해당 job 에 credit assignment 하는 방법이 없다.
- Shared adapter 와 external retrieval 의 break-even 이 hardware·batching 에 따라 크게 달라진다.
- Regulation 이 weight-based personalization 을 external memory 보다 더 엄격히 다룰 가능성이 있다.
- “Sleep” label 이 과도한 anthropomorphism 을 낳아 system boundary 를 흐릴 수 있다.

핵심

주류가 될 가능성이 가장 높은 것은 “모델이 인간처럼 잔다” 는 서사가 아니라, response path 와 background memory maintenance 를 분리하는 operational architecture 다.

더 깊이 읽기

- 원 분석: 조건부 promisingness 와 주류화 scenario
- Training Background: 서버산출물, 외부기억, 모수적기억, 버전관리, 롤백
- Claim route: **STC-C038** · synthesis · confidence high · **STC-C039** · synthesis · confidence medium · **STC-C040** · synthesis · confidence medium · **STC-C042** · synthesis · confidence high

E09 · Sleep-time scaling law 는 어떻게 세워야하는가

이 권이 답할질문

Pretraining 에는 compute·data·parameter 와 loss 의 경험적 scaling law 가있다. Sleep-time compute 에도 “더 오래 자면 더 똑똑해진다” 는 법칙이생길까? 현재 답은 **아직 universal law 가 없다**. 먼저 cost accounting identity, reuse break-even, cadence optimum, capacity knee 를 분리하고, 이후 workload 별 empirical fit 을 쌓아야 한다.

이 권은 scaling-law section 을 풀며 **STC-C041** · synthesis · confidence high, **STC-C043** · hypothesis · confidence medium, **STC-C044** · hypothesis · confidence medium, **STC-C045** · hypothesis · confidence medium 을 구분한다.

먼저 세 문장

1. Raw sleep FLOP, 처리한 memory 수, 생성한 dream token 은 좋은 scaling variable 이 아니다. 잘못된 기억을 많이 처리하면 utility 가 음수가 될 수있다.
2. 핵심 후보는 **valid reusable evidence per unit sleep cost** 다. Validity, reuse, retention, staleness, interference 를 함께 측정해야한다.
3. Sleep cadence 와 memory capacity 에는 interior optimum 이 있을 가능성이높다. 너무 자주 갱신하면 overhead·noise 가 크고, 너무 늦으면 staleness 가 커진다.

직관

Identity 와 law 를구분한다

다음은 정의에 가까운 accounting identity 다.

$$C_{\text{sleep}} = C_{\text{read}} + C_{\text{transform}} + C_{\text{train}} + C_{\text{eval}} + C_{\text{publish}}$$

각 항을 더하면 총비용이 된다는 것은 항상 참이지만 성능이 어떻게변하는지는 말하지 않는다. Scaling law 라면 여러 규모에서 측정한 data 로예측력이 있어야 한다. 예를 들어 utility gap 이 sleep compute 의 power law 로 줄어드는지, 어느 점에서 saturation 하는지 fit 하고 out-of-sample 에서맞아야 한다.

Break-even 은 reuse 문제다

External evidence 를 매번 retrieve 하는 비용을 c_r , 한 번 sleep/compile 하는 비용을 c_s , compile 후 매 query 절약을 Δc 라 하자. 단순 break-even reuse 는 다음처럼 생각할 수 있다.

$$N^* \approx \frac{c_s + c_{\text{governance}}}{\Delta c}$$

$N > N^*$ 일 때만 compile 이 경제적이다. 하지만 quality loss 와 rollback risk 를 cost 에 포함해야 한다. 이 식은 measurement model 이지검증된 universal law 가 아니다.

좋은 분모와 좋은분자

분모에는 GPU FLOP 만 넣지 않는다. Joule, wall-clock, accelerator-hour, HBM/SSD bytes moved, index write amplification, validation inference 를기록한다. 분자에는 raw memory count 대신 future task 에서 실제로 사용된 **valid retained evidence** 를 둔다.

한 후보 metric 은 다음과 같다.

$$\eta_{\text{STC}} = \frac{U_{\text{future}} \times R_{\text{retained}} \times P_{\text{valid}}}{C_{\text{sleep}} + C_{\text{serve overhead}} + C_{\text{rollback}}}$$

여기서 U 는 future utility, R 은 retention, P 는 provenance/validation 을 통과한 비율이다. 이 역시 law 가 아니라 연구를위한 normalization proposal 이다.

Cadence optimum

Sleep interval τ 가 작으면 fresh 하지만 job launch 와 validation 이너무 자주 일어난다. τ 가 크면 batch efficiency 는 좋아지지만 memory 가 stale 해지고 wake state 가 넘칠 수 있다. Traffic 과 update entropy 가 변하므로 optimal τ 는 constant 가 아닐 가능성이 높다.

Capacity knee

Memory byte M 을 늘릴 때 초반 utility 는 증가하지만 중복 ·interference·retrieval noise·compaction cost 때문에 marginal gain 이떨릴 수 있다. 용량예산의 knee 를 넘으면 더 큰 device 보다 admission·eviction policy 가중요해진다.

예와 반례

예 — Adapter compile break-even 실험

동일한 knowledge bundle 을 RAG 로만 제공하는 system 과 adapter 로 compile 한 system 을 비교한다. Reuse 횟수를 1, 10, 100, 1000 으로 바꾸고 answer quality, TTFT, energy, adapter load, delete cost 를 쟀다. Break-even 은 quality constraint 를 만족하는 최초 reuse 지점이다. Hardware 가 바뀌면 N^* 도 바뀐다.

예 — Cadence sweep

Sleep interval 을 1 분, 1 시간, 1 일, 1 주로 바꾸고 staleness, batch size, queue delay, wake SLA interference, derived-memory error 를 측정한다. U-shaped total cost 가 나오면 interior optimum 가설이 지지된다. 단일 cadence 의 SOTA 만 보고 law 라고 부르면 안 된다.

반례 — Dream token 을 10 배 늘렸더니 benchmark +1

한 점의 결과는 scaling law 가 아니다. 다른 model size, data quality, recursive depth, task 에서 반복되고 held-out scale 을 예측해야 한다. 또한 extra token 이 중복이면 유효 evidence 는 거의 늘지 않았을 수 있다.

수식의 지위를 섞지 않는다

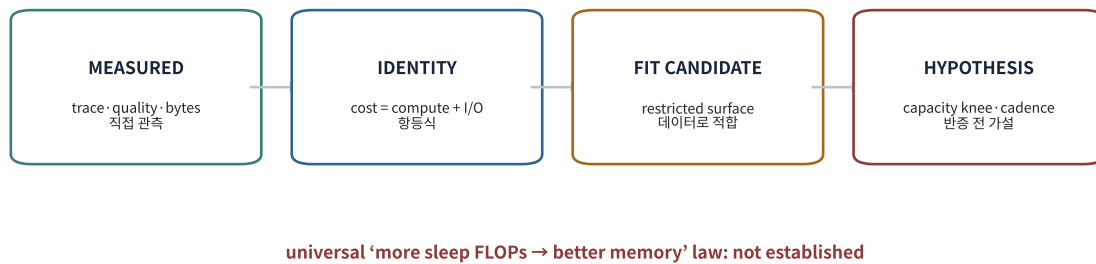


Figure 16: STC-F007. Measured identities, break-even models, and hypotheses. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

반례 — Memory byte 가 크면 capacity 문제해결

Storage capacity 를 늘려도 active index, cache hit, retrieval precision, write endurance 가 병목일 수 있다. 유한한 latency 와 energy 안에서 usable capacity 를 재야 한다. “저장 가능한 byte” 와 “정확히 찾아 활용 가능한기억” 은 다르다.

그림 읽기

Figure 는 measured identity, break-even model, fit candidate, untested hypothesis 를 색으로 분리한다. 수식이 있다는 이유만으로 theory 가 되는것을 막기 위한 장치다. 연구 보고서에는 각 equation 의 status 를 함께적어야 한다.

Sleep cost 는 한 번 앞에서 지불하고 later reuse 에서 절약을 얻는다. Curve 가 0 을 지나는 지점이 break-even 이다. External memory maintenance 처럼 read cost 를 줄이지 않고 quality 를 높이는 경우에는 절약이 아니라 utility gain 을 monetary/energy-equivalent 로 환산해야 한다.

Capacity knee 는 finite-memory system 의 saturation 가설이다. Knee 이전에는 byte 추가가 유용하고, 이후에는 compaction·eviction·tiering 이 더중요하다. Device solution 은 knee 를 오른쪽으로 옮기거나 knee 이후 비용을 낮추는 제품으로 정의할 수 있다.

Cadence frontier 는 staleness 와 batching 의 tradeoff 를 보여 준다. Fixed daily sleep 보다 event-triggered/learned cadence 가 frontier 를 개선할 수있다. 그러나 scheduler 를 학습하려면 delayed future utility 를 현재 sleep decision 에 연결하는 평가가 필요하다.

대안과 비교

후보 관계	현재 지위	필요한 축	반증 조건
sleep FLOP ↑ → utility ↑	미검증	model, data quality, task	동일 compute 에서 saturation/하락
reuse ↑ → compile 가치 ↑	break-even model	access count, latency, quality	governance cost 가 절약 초과

후보 관계	현재 지위	필요한 축	반증 조건
cadence 에 interior optimum	가설	staleness, batch, SLA	boundary 가항상 최적
capacity knee 존재	가설	byte, entropy, interference	marginal utility 가 계속 일정
valid evidence efficiency	metric proposal	utility, validity, cost	measurement 가 불안정

오프폴리시평가는 실제로 다른 cadence 를 배포하지 않고 logged policy 에서 utility 를추정하는 후보지만, hidden confounding 과 distribution shift 에 취약하다. 초기에는 controlled sweep 과 replay simulation 이 필요하다.

아직 모르는것

- Future utility 를 memory write 에 귀속하는 credit assignment 가미해결이다.
- External synthesis 와 parametric training 의 cost unit 을 공정하게통일하기 어렵다.
- Quality·safety·delete 를 하나의 objective 로 결합하면 weight 선택에 따라결론이 달라진다.
- Model size 가 커질 때 sleep sample efficiency 가 좋아지는지 나빠지는지공개 scaling sweep 이 없다.
- Capacity knee 가 device bandwidth, index algorithm, memory entropy 중무엇에 더 민감한지 모른다.

핵심

첫 STC scaling law 는 “sleep FLOP law” 보다 “재사용되는 유효 evidence 와 lifecycle 총비용의 관계” 에서 나올 가능성이 높다.

더 깊이 읽기

- 원 분석: accounting identity·break-even·hypothesis 의 엄격한 구분
- Training Background: 목적함수, 검증세트, 용량예산, 울-왜곡, 오프폴리시평가
- Claim route: **STC-C041** · synthesis · confidence high · **STC-C043** · hypothesis · confidence medium · **STC-C044** · hypothesis · confidence medium · **STC-C045** · hypothesis · confidence medium

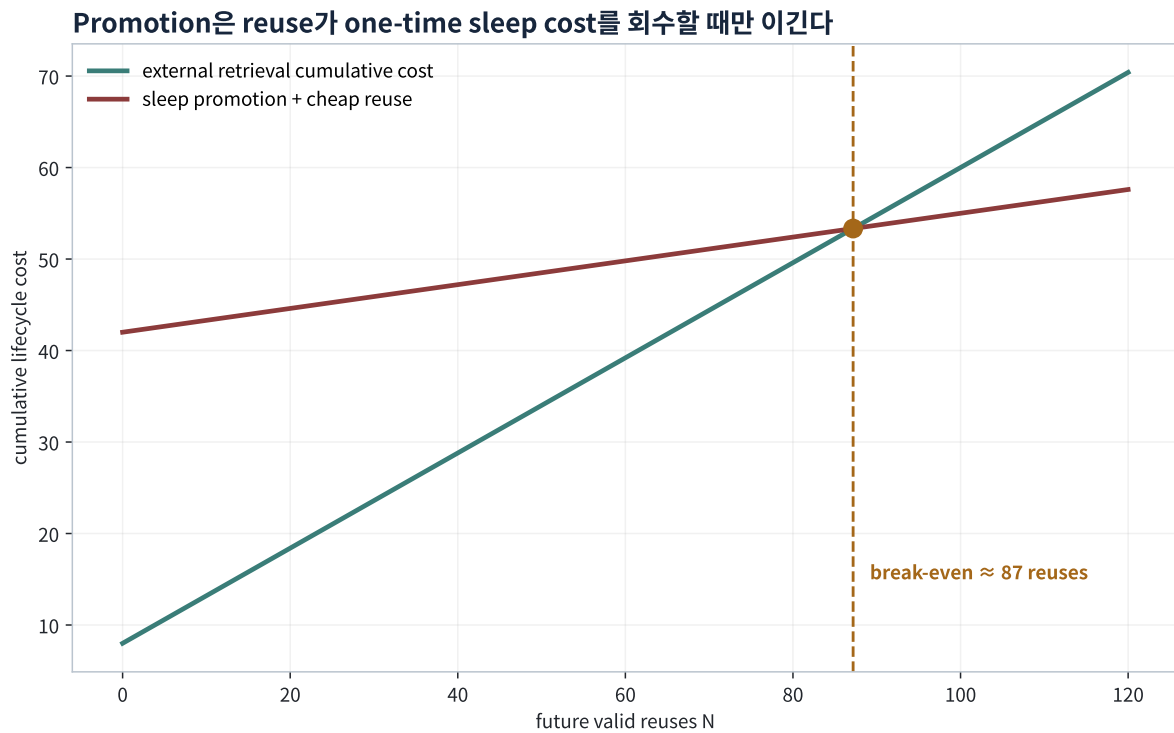


Figure 17: STC-F008. Sleep-cost versus later-reuse break-even. 원 Study 의저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

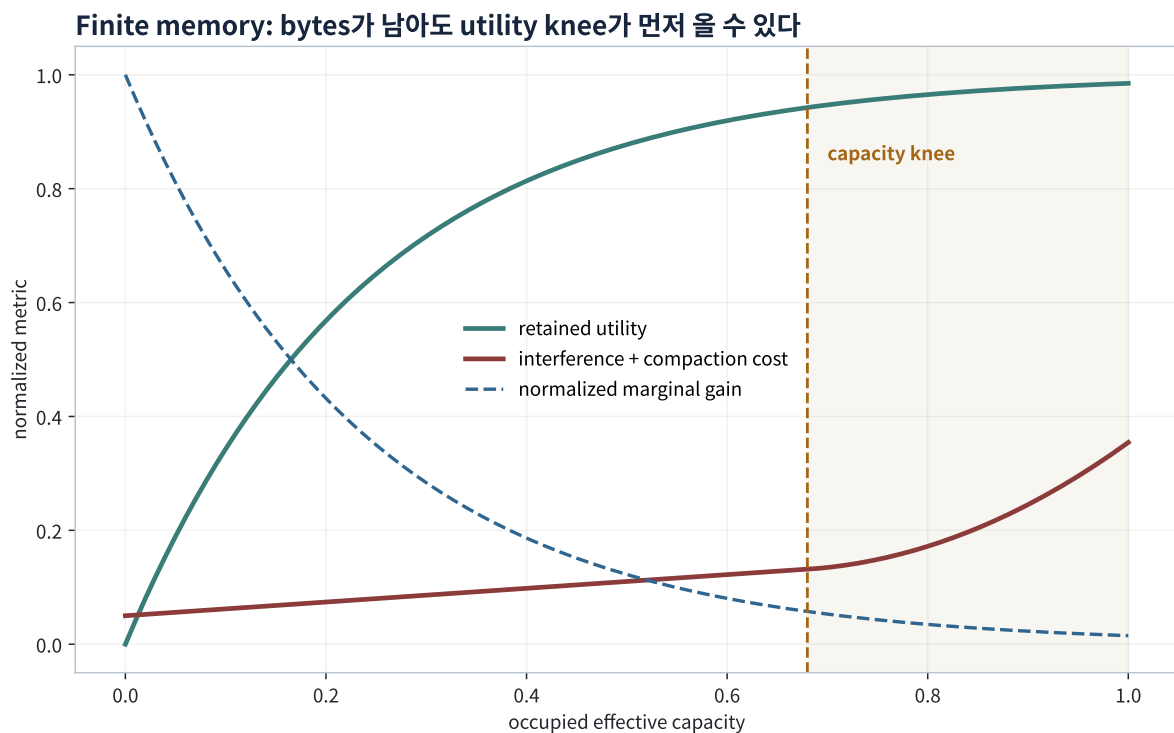


Figure 18: STC-F009. Finite-memory capacity knee. 원 Study 의 저자 작성합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

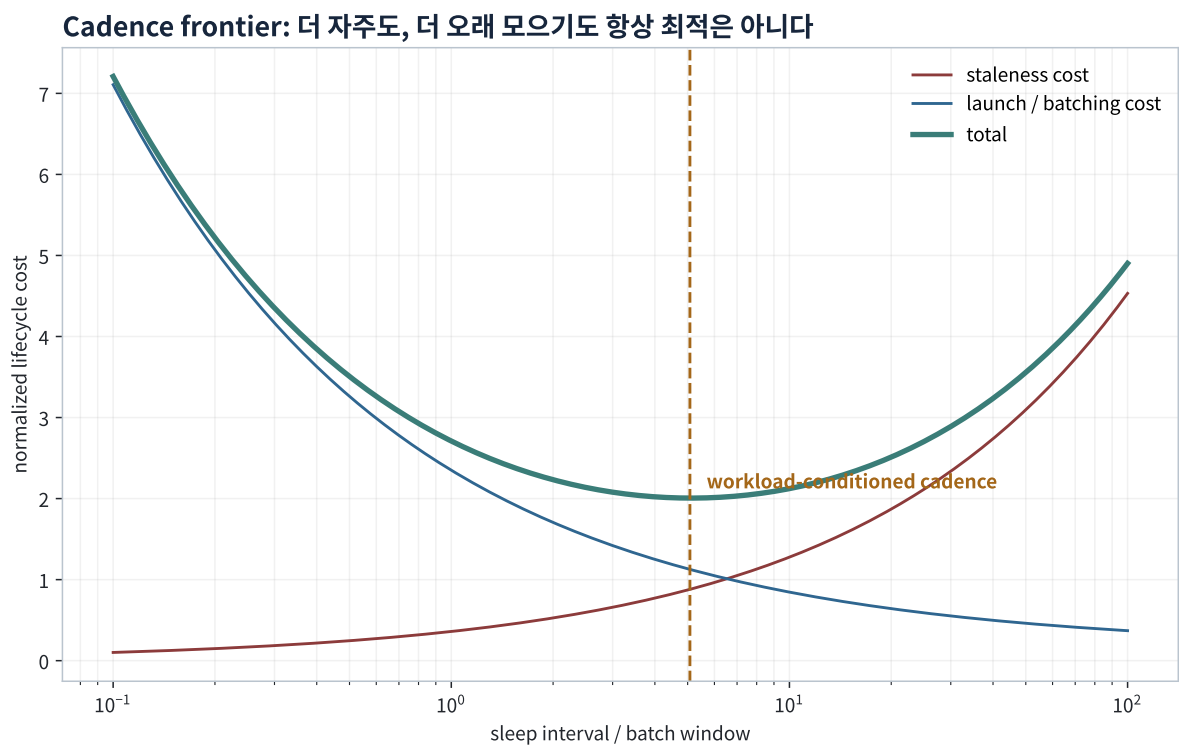


Figure 19: STC-F010. Cadence frontier across staleness and batching. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

E10 · Wake·sleep·long-term memory infra 는 어떻게나뉘는가

이 권이 답할질문

배포 후 학습이 wake 와 sleep 으로 나뉘다면 cluster 와 data plane 도 나뉘어야하는가? 그렇다. 다만 세 개의 독립 silo 를 만드는 것이 아니라 wake SLA, background training, durable memory 가 서로 다른 최적점을 가지므로 **control plane** 과 **versioned state contract** 로 연결해야 한다. Infra 목표는 state movement 를 줄이고, sleep output 을 안전하게 publish 하며, 외부 memory 와 model delta 를 같은 lineage 로 관리하는 것이다.

이 권은 system infrastructure 를 설명한다. 근거는 **STC-C025 · author_claim · confidence high**, **STC-C040 · synthesis · confidence medium**, **STC-C046 · synthesis · confidence high**, **STC-C053 · hypothesis · confidence medium**, **STC-C055 · hypothesis · confidence medium** 다.

먼저 세 문장

1. Wake cluster 는 TTFT·ITL·availability 를, sleep cluster 는 throughput·sample efficiency·batching 을 최적화한다. 같은 GPU 를 써도 scheduler objective 가 다르다.
2. Long-term memory fabric 은 episode, vector, graph, recurrent state, adapter, optimizer snapshot 을 tiering 하고 provenance·version·delete lineage 를 유지한다.
3. Wake 와 sleep 사이의 핵심 interface 는 raw log dump 가 아니라 **immutable evidence ID + state generation + compatibility/version metadata** 다.

직관

Wake cluster

Wake path 는 사용자가 기다린다. Model weights, active adapter, KV/recurrent state, hot retrieval index 가 low-latency tier 에 있어야한다. Online update 를 허용하면 inference 만이 아니라 read-modify-write 가 생긴다. 하지만 긴 backward 나 validation 을 response path 에 넣으면 tail latency 가 무너진다.

Sleep cluster

Sleep path 는 여러 user/session 의 job 을 batch 하고 cheap/off-peak accelerator 를 쓸 수 있다. Teacher rollout, embedding, graph compaction, LoRA training, regression evaluation 이 섞인다. High throughput 이 중요하지만 한 job 의 잘못된 output 이 많은 future response 에 영향을 주므로 isolation 과 reproducibility 가 필요하다.

Long-term memory fabric

Memory medium 마다 access pattern 이 다르다.

- Episode log: append-heavy, immutable, cold-to-warm scan
- Vector index: embedding read, incremental insert, compaction
- Temporal graph: small random read/write, lineage traversal
- Recurrent/neural state: dense tensor snapshot, prefix keying
- Adapter: versioned blob, read-mostly serving, occasional write

- Optimizer/checkpoint: large sequential write/read, sleep job scoped

하나의 HBM 에 모두 넣는 것은 비경제적이다. Hot state 만 accelerator near-memory 에 두고, warm metadata·index 와 cold evidence 를 tiering 해야한다.

Control plane

버전관리은 “model v3” 하나로 끝나지 않는다. Base model, tokenizer, adapter, memory schema, index build, graph snapshot, policy, evaluator version 의 compatible tuple 이 필요하다. 롤백은이 tuple 의 active pointer 를 이전 generation 으로 원자적으로 돌리는일이다.

Recurrent state cache

Full attention serving 은 token prefix 가 같으면 KV prefix cache 를재사용한다. RNN/HOPE/Titans-like model 은 prefix 를 압축한 state 를재사용해야 한다. **STC-C025** · author_claim · confidence high 가지적하듯 state 를 materialize·key·share 하지 않으면 architecture 의 linear-time 장점이 serving 에서 사라질 수 있다. Cache key 에는 model/adaptor version, exact prefix hash, state schema 가 들어가야 한다.

시스템 모델링 관점

State migration byte 가 compute FLOP 보다 큰 구간이 있다. Sleep job 이 adapter 를 만들기 위해 remote episode 와 checkpoint 를 모두 GPU 로 옮기면 near-data preprocessing 의 가치가 커진다.

예와 반례

한 sleep generation 의 publish protocol

1. Control plane 이 evidence snapshot ID 와 policy/evaluator version 을고정한다.
2. Job scheduler 가 graph/index job 과 GPU training job 을 DAG 로 실행한다.
3. Output 을 new generation namespace 에 쓰고 old generation 을 건드리지않는다.
4. Regression, deletion, poisoning, cost gate 를 실행한다.
5. 통과하면 active pointer 를 atomic switch 한다.
6. Canary metric 이 나빠지면 이전 pointer 로 즉시 rollback 한다.
7. Retention period 뒤에 unreachable generation 을 garbage collect 한다.

이 protocol 은 database compaction 과 model deployment 를 결합한 형태다.

Data movement 를 줄이는방법

- Episode 와 graph close-to-storage 에서 dedup·filter 해 accelerator 로보내는 token 을 줄인다.
- Base model 은 sleep GPU 에 상주시키고 user/domain adapter job 만이동한다.
- Adapter snapshot 은 copy-on-write 로 만들고 unchanged block 을 공유한다.
- Recurrent state 는 wake region 에서 reusable cache 로 materialize 하고 remote migration 을 최소화한다.
- Evaluation dataset 과 teacher logits 를 content-addressed cache 로재사용한다.

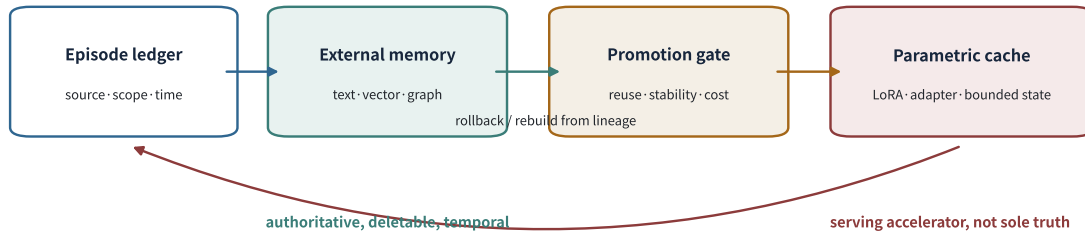
Hybrid: external system-of-record → selective reversible promotion

Figure 20: STC-F006. Hybrid system-of-record and selective-promotion architecture. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

반례 — Wake 와 sleep 이 같은 mutable weight 를 동시에쓰

Inference 가 읽는 중 training 이 같은 weight 를 갱신하면 reproducibility 와 batching 이 깨진다. Mutable in-place update 대신 generation 을 분리하고 publish boundary 에서 pointer 를 바꿔야 한다.

반례 — 모든 user state 를 HBM 예상주

Active user 분포는 long tail 이다. HBM capacity 를 늘려도 cold state 가 hot state 를 밀어내면 효율이 낮다. Access frequency, state size, regeneration cost 로 tier 를 정해야 한다.

그림 읽기

Architecture 의 중심은 external source-of-record 와 selective promotion 이다. Wake cluster 는 derived state 를 읽고, sleep cluster 는 evidence 에서 새 derived state 를 만든다. 화살표는 one-way copy 가 아니라 provenance link 와 invalidation path 를 뜻한다.

State-migration roofline 은 x 축 byte movement 와 y 축 compute 를 놓는다. Graph/index compaction 은 bandwidth·random write bound, adapter training 은 compute/HBM bound, checkpoint publish 는 sequential bandwidth bound 일 수 있다. Device solution 은 workload 를 한 숫자 “AI bandwidth” 로 묶지 말고 phase 별 operational intensity 를 봐야 한다.

Failure control 은 control plane 의 요구사항이다. Poisoned evidence 를 걸러도 derived adapter 가 이미 publish 됐다면 lineage 를 따라 invalidation 하고 rebuild 해야 한다. Observability 에는 answer metric 뿐 아니라 state generation, evidence coverage, rollback latency 가 포함된다.

대안과 비교

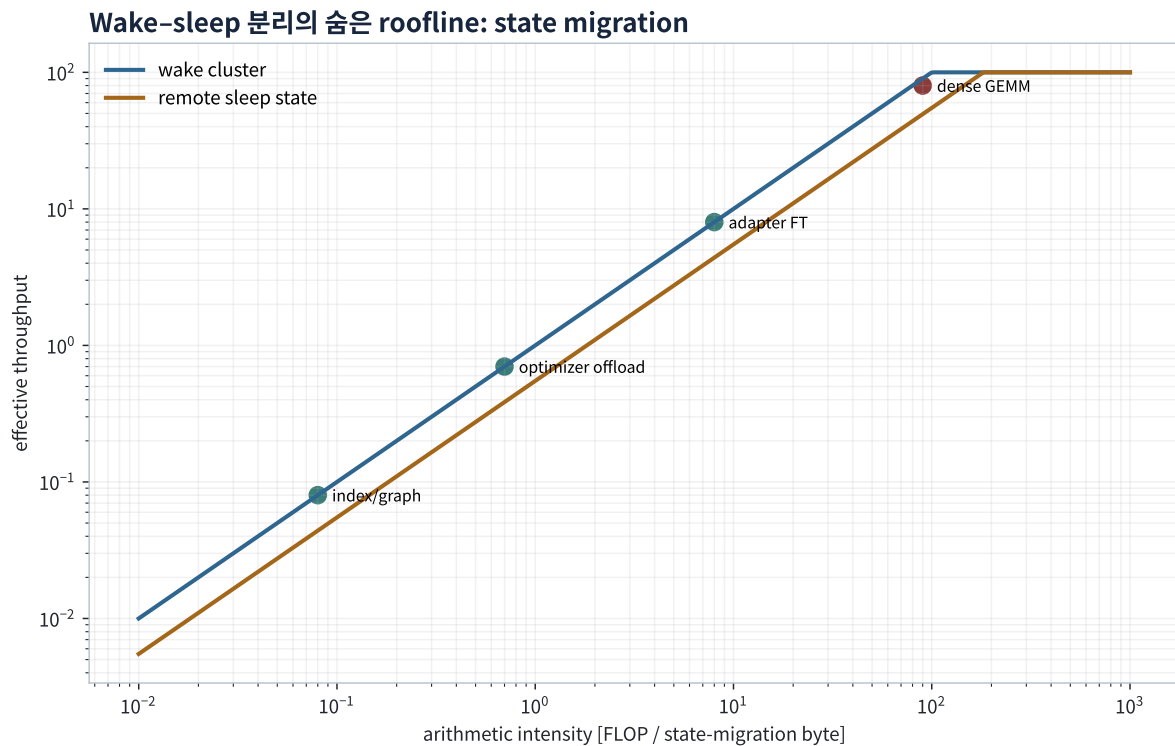


Figure 21: STC-F011. State-migration roofline for wake and sleep clusters. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

Failure mode마다 lifecycle control을 붙인다

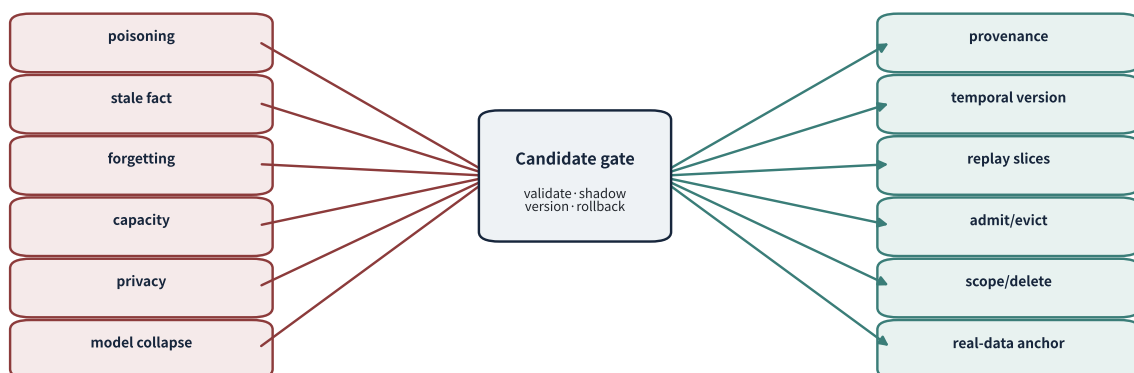


Figure 22: STC-F013. Failure modes and lifecycle controls. 원 Study 의 저자작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

plane	SLO	hot state	주 병목	필수 primitive
wake inference	TTFT/ITL, availability	weights, KV/state, hot index	HBM/BW, batching	cache, routing
wake learning	bounded latency	fast state, accumulator	RMW, interference	isolated delta
sleep compute	throughput, utility	training activations, batch data	GPU compute/data feed	DAG, checkpoint
memory fabric	durability, lookup	tier 별 working set	random write/migration	index, tiering
control plane	correctness	metadata/lineage	version explosion	atomic publish/rollback

샤딩은 user 나 entity 를 분산하지만 cross-user/domain consolidation 이 필요할 때 shuffle 이 생긴다. Reuse 와 privacy boundary 가 같은 shard key 가 되도록 설계하면 이동을 줄일 수 있다.

아직 모르는것

- Wake/sleep cluster 를 물리 분리할지 elastic shared pool 로 운영할지 workload evidence 가 부족하다.
- Recurrent state cache 의 real trace hit ratio 와 eviction policy 가 공개되지 않았다.
- Adapter·graph·vector 를 하나의 transaction 으로 publish 하는 표준이 없다.
- Sleep job 의 queue delay 와 memory staleness 가 user utility 에 미치는 관계가 측정되지 않았다.
- Delete request 가 derived weight 까지 전파되는 rebuild SLA 와 비용이 불명확하다.

핵심

STC infra 의 핵심 제품은 “밤에 쓰는 GPU” 가 아니라 evidence 에서 derived state 를 만들고, 최소 이동으로 검증·publish·rollback 하는 lifecycle fabric 이다.

더 깊이 읽기

- 원 분석: wake/sleep/memory/control plane architecture
- Training Background: 샤딩, 버전관리, 롤백, 기억라우팅, 관측가능성, 서빙산출물
- Claim route: **STC-C025** · author_claim · confidence high · **STC-C040** · synthesis · confidence medium · **STC-C046** · synthesis · confidence high · **STC-C053** · hypothesis · confidence medium · **STC-C055** · hypothesis · confidence medium

E11 · Memory device 회사에 생기는 기회

이 권이 답할질문

Sleep-time compute 가 확산되면 memory device solution 회사는 단순히 더 큰 HBM·SSD 를 파는 것 외에 무엇을 할 수 있는가? 기회는 새 FLOP 수요보다 **derived memory lifecycle** 의 **byte movement, random rewrite, snapshot, provenance, delete/rebuild** 에서 더 선명하다. Device 는 저장매체와 software primitive 를 함께 제안해야 한다.

이 권은 device opportunity map 을 설명하며 **STC-C048** · hypothesis · confidence medium, **STC-C049** · hypothesis · confidence medium, **STC-C050** · hypothesis · confidence medium, **STC-C051** · hypothesis · confidence low, **STC-C052** · hypothesis · confidence medium, **STC-C053** · hypothesis · confidence medium, **STC-C054** · hypothesis · confidence medium, **STC-C055** · hypothesis · confidence medium, **STC-C056** · hypothesis · confidence low, **STC-C057** · hypothesis · confidence medium 을 제품 가설로 변환한다.

먼저 세 문장

1. Sleep workload 는 sequential checkpoint 뿐 아니라 vector/graph metadata 의 작은 random rewrite 와 versioned adapter read 가 크다. Endurance·index locality·copy-on-write 가 중요하다.
2. 가장 차별화 가능한 solution 은 provenance-first store, snapshot/rollback, near-data preprocessing, adapter fabric, reusable-state cache, secure delete/rebuild 다.
3. 평가 지표는 raw bandwidth 가 아니라 **retained useful evidence, migration byte, write amplification, rollback time, wake-SLA interference** 여야 한다.

직관

Opportunity 1 — Provenance-first append-only store

Episode 와 derived memory 를 content-addressed object 로 저장하고 “어느 evidence 와 transform 에서 만들어졌는가” 를 edge 로 남긴다. Append-only generation 은 in-place corruption 을 줄이고 rollback 을 쉽게 한다. 장치는 small metadata lookup 과 sequential object write 를 함께 최적화할 수 있다.

Opportunity 2 — Snapshot 과 copy-on-write

Sleep job 마다 full model·index·graph 를 복제하면 byte 가 폭증한다. Unchanged block 을 공유하고 changed delta 만 쓰는 copy-on-write snapshot 은 버전관리과 롤백을 싸게 만든다. Atomic root pointer switch 는 publish primitive 가 된다.

Opportunity 3 — High-endurance indexed tier

Vector insert, graph edge update, supersede metadata, decay score 는 작은 random write 를 반복한다. HBM peak bandwidth 보다 endurance, write amplification, tail latency 가 중요하다. Log-structured layout, compaction offload, metadata cache 가 solution 이 될 수 있다.

Opportunity 4 — Near-data preprocessing

Raw episode 를 모두 GPU 로 보내기 전에 dedup, filtering, tokenization, embedding candidate search, graph neighborhood gather 를 storage 근처에서 수행한다. 이 기능은 expensive accelerator token 과 interconnect byte 를 줄인다. **STC-C051** · hypothesis · confidence low 은 near-data 를 training accelerator 대체가 아니라 data reduction stage 로 본다.

Opportunity 5 — Versioned adapter fabric

Adapter 는 full model 보다 작지만 user/domain 수가 많아지면 active set 관리가 어렵다. Adapter store 가 version, tenant, base-model compatibility, hotness 를 index 하고 GPU cache 에 prefetch·merge·evict 하면 serving fragmentation 을 줄일 수 있다.

Opportunity 6 — Reusable-state cache

RNN/neural-memory model 은 prefix 를 compressed state 로 materialize 한다. Cache 가 model version+prefix hash 로 state 를 재사용하면 full prefill 을 피할 수 있다. Device 는 KV 뿐 아니라 dense recurrent state, optimizer-like accumulator, memory block snapshot 을 지원해야 한다.

Opportunity 7 — Secure delete-and-rebuild

Source episode 가 삭제되면 summary, vector, graph, adapter 까지 lineage 를 따라 invalidate 해야 한다. Device/firmware 가 cryptographic erase, generation tombstone, fast rebuild scan 을 제공하면 compliance SLA 가 제품 차별점이 된다.

핵심

Memory device 기회는 capacity 판매가 아니라 “기억 generation 의 lifecycle transaction” 을 빠르고 검증 가능하게 만드는 데 있다.

예와 반례

Device solution package 예

1. **Evidence Log Tier** — immutable append, encryption, content hash
2. **Derived Index Tier** — high-endurance vector/graph update, compaction engine
3. **Adapter Vault** — versioned small tensor object, compatibility metadata
4. **Hot State Cache** — KV/recurrent/neural state unified caching
5. **Snapshot Engine** — COW generation, atomic publish, rollback
6. **Lineage/Deletion Engine** — provenance traversal, tombstone, rebuild queue
7. **Telemetry** — byte moved, WA, cache hit, rollback, staleness counters

이는 단일 chip 보다 controller, firmware, runtime, reference architecture 의 결합이다.

On-device sleep 예

Smartphone 이 charging·Wi-Fi·thermal 여유 상태에서 private episode 를 local summary 로 만들고, small personalization adapter 를 업데이트한다. Raw data 는 device 를 떠나지 않는다.

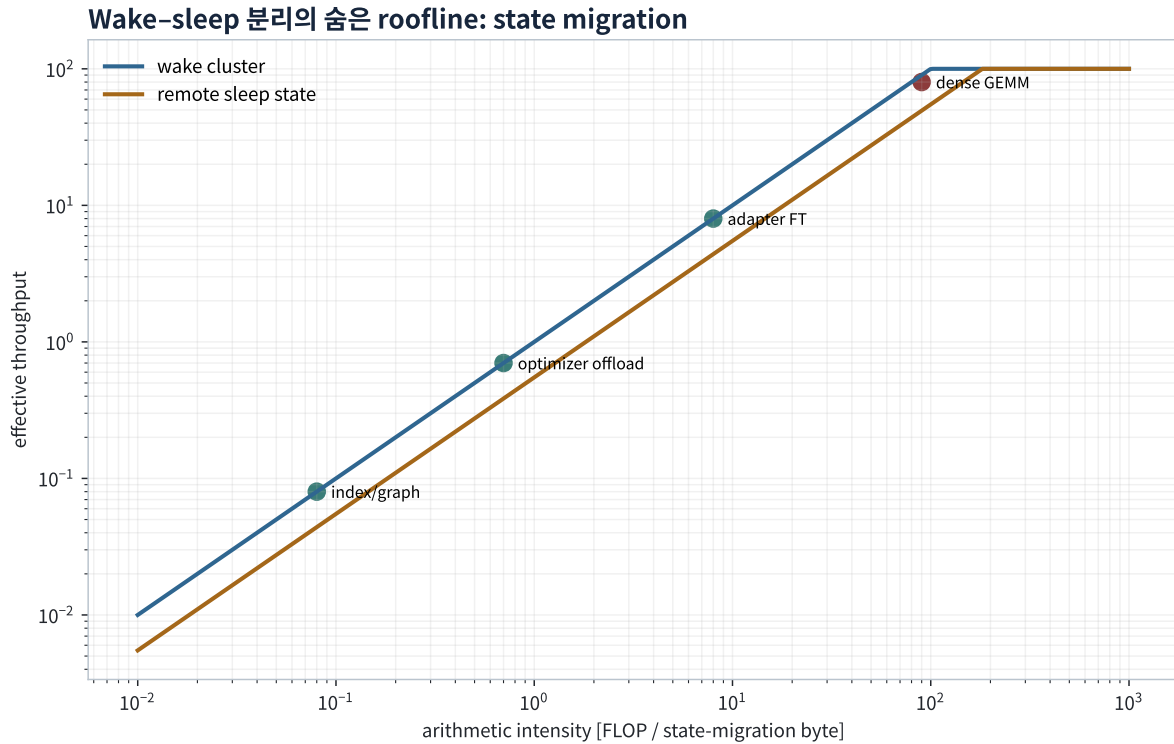


Figure 23: STC-F011. State-migration roofline for wake and sleep clusters. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

NPU/DRAM/flash 사이 이동, flash endurance, encrypted snapshot, 실패 시 rollback 이 중요하다. 모든 task 가 아니라 privacy 가치가 큰 개인 assistant 에서 유망하다.

반례 — HBM 용량만 2 배

Working set 이 adapter·index·graph·episode 로 섞여 있고 access 가 sparse 하면 HBM 에 모두 올리는 것은 비싸다. Capacity 가 늘어도 cold object, random write, version metadata 문제는 남는다. Tiering policy 와 object semantics 가 없으면 usable utility 는 선형 증가하지 않는다.

반례 — GPU 근처에 모든 preprocessing 배치

Dedup 할 raw episode 를 먼저 GPU 까지 옮기면 interconnect 와 HBM bandwidth 를 이미 썼다. Storage-side filter 가 90%를 제거할 수 있다면 near-data 가 더합리적이다. 반대로 deep semantic evaluator 는 GPU 가 필요하므로 function placement 를 workload 별로 나눠야 한다.

그림 읽기

Roofline 의 각 점은 다른 device 병목을 가진다. Teacher/student training 은 compute-dense, graph maintenance 는 bandwidth/random-write, snapshot 은 sequential capacity, adapter serve 는 latency/cache hit 가 지배한다. “Sleep accelerator” 하나보다 heterogeneous tier 를 연결하는 것이 중요하다.

Opportunity map 은 가까운 기회와 먼 가설을 나눈다. Snapshot, provenance store, indexed tier, adapter cache 는 기존 storage primitive 의 확장으로 시작할 수 있다. Learned in-memory consolidation 이나 autonomous on-device dreaming 은 연구 위험이 더 높다. SAIT/메모리 팀 은 가까운 primitive 부터 trace 와 benchmark 를 확보할 수 있다.

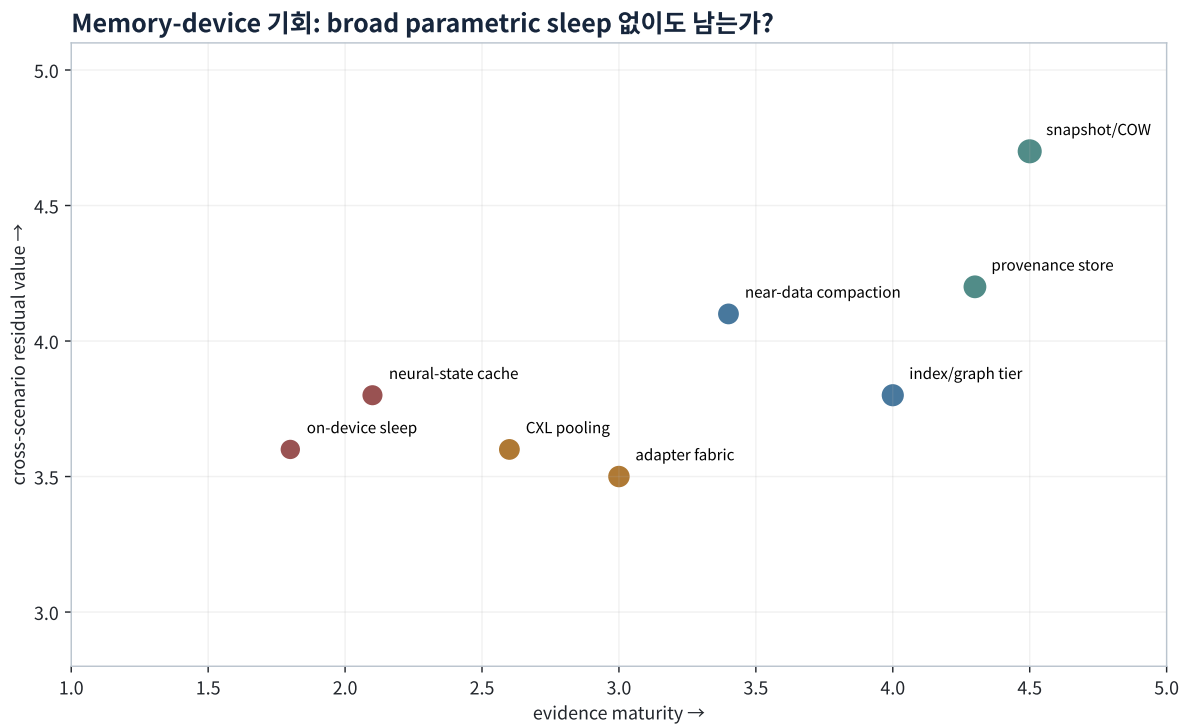


Figure 24: STC-F012. Memory-device opportunity map. 원 Study 의 저자 작성합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

대안과 비교

solution	고객 문제	device primitive	KPI	first demo
provenance store	audit/rebuild	append+lineage index	lineage query latency	memory delete trace
COW snapshot	safe publish	block sharing+atomic root	snapshot byte/time	adapter generations
indexed endurance tier	graph/vector rewrite	log structure+compaction	WA/endurance	temporal KG stream
near-data preprocessing	state migration	filter/dedup/-gather	bytes avoided/J	episode synthesis
adapter fabric	version explosion	object index+prefetch	hit/batching gain	multi-tenant LoRA
reusable-state cache	RNN prefix reuse	state key+cache	state hit/TTFT	HOPE/NSTM serving
delete/rebuild engine	compliance	tomb-stone+scan	delete SLA/leakage	derived-memory DAG

옵티마이저상태도 sleep checkpoint 에서 weight 보다 2 배 이상 클 수 있어 sharded snapshot 과 cold tier 가 필요하다. 다만 inference-only adapter 는 optimizer state 를 publish 하지 않아도 되므로 train-time object 와 serve-time object 를 분리해야 한다.

아직 모르는것

- 실제 production trace 에서 graph/vector/adaptor/episode byte 비율과 hotness distribution 이 공개되지 않았다.
- Near-data semantic filtering 의 accuracy loss 와 accelerator byte 절감 tradeoff 가 측정 되지 않았다.
- Recurrent state cache 가 어떤 granularity 에서 KV cache 보다 경제적인지 workload 가 부족하다.
- Adapter merge 가 quality 와 batch efficiency 에 미치는 영향이 model 별로 다르다.
- Secure delete 가 parametric artifact 에 대해 “완료” 되었다고 증명하는방법이 없다.

시스템 모델링 관점

Device roadmap 은 speculative STC FLOP forecast 보다, 오늘 구현 가능한 lifecycle trace 를 수집하고 state-migration roofline 을 만드는 것에서 시작해야 한다.

더 깊이 읽기

- 원 분석: 10 개 memory-device opportunity 와 maturity
- Training Background: adapter, 옵티마이저상태, 샤딩, 버전관리, 롤백, 출처추적성
- Claim route: **STC-C048** · hypothesis · confidence medium · **STC-C049** · hypothesis · confidence medium · **STC-C050** · hypothesis · confidence medium · **STC-C051** · hypothesis · confidence low · **STC-C052** · hypothesis · confidence medium · **STC-C053** · hypothesis · confidence medium · **STC-C054** · hypothesis · confidence medium · **STC-C055** · hypothesis · confidence medium · **STC-C056** · hypothesis · confidence low · **STC-C057** · hypothesis · confidence medium

E12 · 무엇을 측정하면 이 가설을 믿거나 버릴 수 있는가

이 권이 답할질문

Sleep-time compute 를 research program 으로 밀기 전에 어떤 실험이 성공해야하고, 어떤 결과가 나오면 방향을 버리거나 축소해야 하는가? Promising 한가설은 반증 조건을 가져야 한다. 이 권은 benchmark, staged roadmap, failure control, go/no-go rule 을 하나로 묶는다.

원문 경로는 benchmark roadmap, limitations and falsifiers, conclusion 이다. 근거는 **STC-C035** · author_claim · confidence high, **STC-C036** · author_claim · confidence high, **STC-C037** · synthesis · confidence high, **STC-C039** · synthesis · confidence medium, **STC-C041** · synthesis · confidence high, **STC-C057** · hypothesis · confidence medium 이다.

먼저 세 문장

1. Benchmark 는 current accuracy 가 아니라 retention, access, provenance, deletion, poisoning, cost, wake-SLA 를 함께 재야 한다.
2. 연구는 external synthesis baseline → selective adapter promotion → learned cadence → long-horizon fleet simulation 순으로 단계화한다. 앞 단계가실패하면 뒤 단계로 가지 않는다.
3. Sleep 이 동일 budget 의 retrieval/replay/adapter baseline 보다 utility per cost 에서 이기지 못하거나 rollback·delete 를 만족하지 못하면 broad deployment 가설은 반증된다.

직관

Benchmark unit 은 “기억 하나” 가아니다

하나의 memory record 가 여러 future query 에 다른 방식으로 쓰인다. 따라서 benchmark episode 에는 source, valid time, sensitivity, expected reuse, contradiction, delete event 가 함께 있어야 한다. Evaluation 은 write 직후가 아니라 여러 sleep cycle 뒤에도 반복한다.

Metric family

- **Utility**: future task success, answer accuracy, strategy reuse
- **Retention**: old episode/task performance, backward transfer
- **Access**: encoded information 이 실제 answer 에서 회수되는 비율
- **Freshness**: change 후 stale answer 지속 시간
- **Provenance**: answer/derived memory 가 source 로 연결되는 비율
- **Deletion**: source delete 후 external·parametric leakage
- **Robustness**: poisoned memory 의 spread 와 containment
- **Systems**: sleep joule, bytes moved, write amplification, TTFT/ITL impact
- **Recovery**: rollback time, rebuild time, lost generation 수

관측가능성은 이 metric 을 state generation 과 연결하는 telemetry 다. “평균 score 가 내려갔다” 가 아니라 어느 sleep job 과 evidence 가 원인인지 찾아야한다.

Four-stage roadmap

Stage 1 — External baseline. Immutable episode, temporal graph, vector index, background synthesis 를 구현한다. Provenance·delete·correction 이 되는 기준선을 만든다.

Stage 2 — Selective promotion. High-reuse stable subset 만 adapter 로 compile 하고 RAG 와 break-even 을 비교한다. Adapter 는 external record 에서 재현 가능해야 한다.

Stage 3 — Adaptive cadence. Fixed schedule sweep 으로 utility/staleness/cost curve 를 얻은 뒤 event-triggered 또는 learned scheduler 를 시험한다.

Stage 4 — Long-horizon simulation. 수백 generation, user churn, concept drift, poisoning, deletion 을 포함해 capacity knee 와 lifecycle economics 를 측정한다.

예와 반례

최소 viable benchmark stream

1. 10 만 episode 를 시간순으로 흘린다.
2. 일부는 stable fact, 일부는 preference, 일부는 transient request, 일부는 poison 이다.
3. Fact change 와 delete request 를 중간에 넣는다.
4. 매 interval 에 future query 와 unrelated control task 를 평가한다.
5. External-only, replay, EWC/GEM, adapter, hybrid sleep 을 같은 byte/compute budget 으로 비교한다.
6. 100 회 이상 generation 에서 cumulative error 와 recovery 를 기록한다.

숫자는 illustrative benchmark scale 이며 universal requirement 가 아니다. 핵심은 short static test 가 아니라 time-ordered lifecycle 을 만드는것이다.

Go condition

Hybrid sleep 이 external-only 보다 같은 quality 에서 later-query cost 를 줄이고, adapter-only 보다 provenance·delete 를 유지하며, wake SLA 영향없이 positive net utility 를 보이면 다음 단계로 간다.

No-go condition

- Derived memory error 가 cycle 마다 누적되고 correction 으로 회복되지 않는다.
- 동일 budget 의 RAG/replay baseline 보다 retained utility 가 낮다.
- Delete request 후 parametric leakage 를 허용 기준 아래로 내릴 수 없다.
- Adapter/version fragmentation 이 serving gain 을 상쇄한다.
- Sleep job 이 wake peak 와 경쟁해 SLO 를 지속적으로 위반한다.

반례 — Synthetic collapse 는 항상일어난다

STC-C035 · author_claim · confidence high 는 real-data support 가 사라진 recursive training 의 위험을 보여 주지만, STC-C036 · author_claim · confidence high 은 적절한 real/synthetic accumulation 에서 collapse 가 필연은 아님을 한정한다. 따라서 benchmark 는 “synthetic 인가 아닌가” 보다 recursive depth, anchor ratio, tail retention 을 sweep 해야 한다.

Failure mode마다 lifecycle control을 붙인다

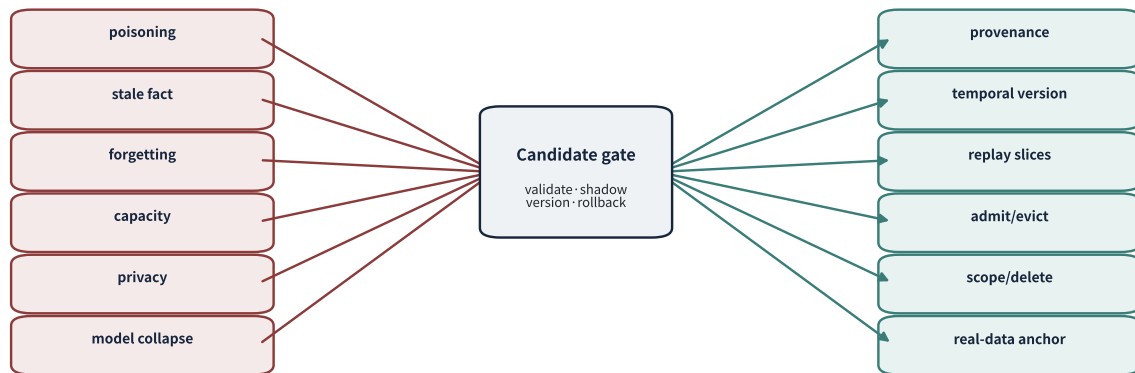


Figure 25: STC-F013. Failure modes and lifecycle controls. 원 Study 의 저자작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

반증 가능한 research → system → device agenda

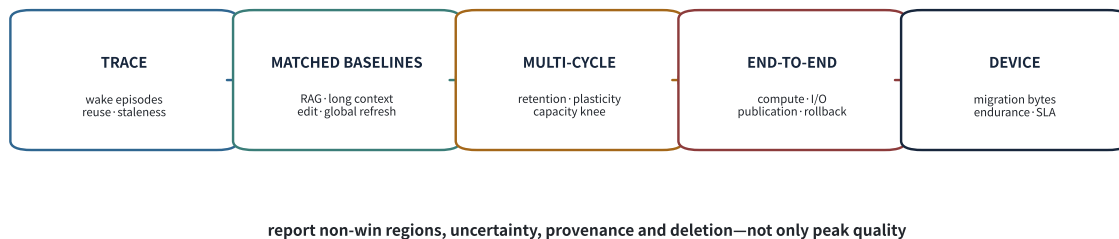


Figure 26: STC-F015. Falsifiable research and benchmark agenda. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

반례 — 한 번의 delete test 통과

External record 에서 삭제됐어도 summary cache, graph edge, adapter, backup generation 에 남아 있을 수 있다. 머신러닝과 lineage-driven rebuild 를 함께 검증해야 한다. Delete completeness 는 artifact 전체의 property 다.

그림 읽기

Failure-mode map 은 benchmark injection point 를 준다. Ingest poisoning, summary hallucination, replay collapse, adapter interference, version mismatch, stale serve, incomplete delete 를 각각 의도적으로 넣고 detection · containment · recovery 를 측정한다.

Research agenda 는 질문을 measurement 와 falsifier 로 연결한다. “주류가 될까?” 같은 큰 질문을 capacity knee, break-even reuse, cadence frontier, delete leakage, state migration 으로 분해한다. 각 연구 package 는 성공 score 와 중단 threshold 를 가져야 한다.

대안과 비교

연구 package	baseline	핵심 metric	go	stop/falsifier
external synthesis	raw RAG/KG	utility, correction	conflict 감소	hallucinated merge 증가
adapter promotion	RAG, LoRA-only	break-even reuse	net cost 절감	delete/locality 실패
cadence	fixed intervals	staleness+cost	interior gain	boundary 가항 상 우세
capacity	larger store	utility/byte	knee 이동	noise 가 gain 상쇄
device tiering	flat HBM/SSD	bytes, WA, SLA	migration 감소	software overhead 우세
long horizon	short benchmark	cycle drift	stable 100+ gen	error 단조 누적

오프폴리시평가는 cadence 후보를 싸게 거를 수 있지만 final go/no-go 에는 live or controlled intervention 이 필요하다. 출판게이트는 claim 을 direct fact, synthesis, hypothesis 로 표시하고 falsifier 가 없는 scaling-law 표현을 막는다.

아직 모르는것

- Human correction cost 를 automated metric 과 어떻게 합칠지 모른다.
- User privacy 와 personalization utility 를 동일 benchmark 에서 비교할 consent framework 가 없다.
- Parametric delete completeness 를 black-box behavior 만으로 증명할 수있는지 불명확하다.
- Long-horizon simulation 이 실제 user behavior drift 를 얼마나 대표하는지모른다.
- Device trace 가 공개되지 않으면 infra scaling hypothesis 를 independent 하게 검증하기 어렵다.

핵심

Sleep-time compute 를 믿는 가장 좋은 방법은 멋진 sleep metaphor 가 아니라, strongest baseline 과의 시간순 benchmark 에서 이길 조건과 질 조건을 미리적어 두는 것이다.

더 깊이 읽기

- 원 분석: benchmark roadmap, limitations·falsifiers, 조건부결론
- Training Background: 검증세트, 오프폴리시평가, 머신언러닝, 출판게이트, 관측가능성, 출처추적성
- Claim route: **STC-C035** · author_claim · confidence high · **STC-C036** · author_claim · confidence high · **STC-C037** · synthesis · confidence high · **STC-C039** · synthesis · confidence medium · **STC-C041** · synthesis · confidence high · **STC-C057** · hypothesis · confidence medium

문서 경계와 사용법

이 문서는 쉬운 설명본이지 새로운 증거원이 아니다. 수치 · 인용 · claim status 는 원 Study paper 와 claim map 이 우선한다. 학습 개념은 Training Background paper 를 함께 읽고, 논문 재현이나 외부 배포 판단에는 고정 source 와 rights audit 를 사용한다.